



Universidade de Brasília - UNB

Faculdade de Tecnologia - FT

Departamento de Engenharia Elétrica - ENE

Programa de Pós-Graduação em Engenharia Elétrica - PPEE

**MAS-Hunt: Arquitetura de Governança
Multiagente Resiliente para a Busca Proativa
por Ameaças Cibernéticas em Ambientes
Windows**

Paulo Matheus Nicolau Silva

Área de concentração: Segurança Cibernética

Linha de pesquisa: Segurança e inteligência Cibernética.

Tema: Arquitetura de Governança para Agentes de Inteligência Artificial na
Busca Proativa por Ameaças Cibernéticas.

Brasília 2025

Universidade de Brasília — UnB
Faculdade de Tecnologia — FT
Departamento de Engenharia Elétrica — ENE
Programa de Pós-Graduação em Engenharia Elétrica — PPEE

Paulo Matheus Nicolau Silva

**MAS-Hunt: Arquitetura de Governança Multiagente
Resiliente para a Busca Proativa por Ameaças
Cibernéticas em Ambientes Windows**

Dissertação de mestrado apresentada ao Programa de Pós-Graduação em Engenharia Elétrica da Universidade de Brasília, como requisito parcial à obtenção do título de Mestre em Engenharia Elétrica, na área de concentração Segurança Cibernética.

Área de concentração: Segurança Cibernética.

Linha de pesquisa: Segurança e Inteligência Cibernética.

Orientador(a): a definir.

Brasília
2025

Resumo

A crescente sofisticação e a natureza evasiva das ameaças cibernéticas contemporâneas — marcadas pelo uso de técnicas de *living-off-the-land* (LOLBins), pela permanência prolongada de atacantes (*dwell time*) e pela sobrecarga cognitiva imposta aos analistas de segurança — expõem os limites das arquiteturas reativas e baseadas em assinaturas. Agentes autônomos apoiados em grandes modelos de linguagem (LLMs) surgem como alternativa proativa, porém são, eles próprios, vulneráveis a manipulação adversarial, incluindo envenenamento de memória, injeção de *prompt* e amplificação de mau funcionamento. Esta dissertação apresenta o MAS-Hunt (*Multi-Agent System for Threat Hunting*), uma arquitetura multiagente em três camadas — Conselho de Governança, Gerentes e Trabalhadores/Caçadores — concebida com prioridade à segurança e materializada como um *plugin* do Claude Code que orquestra subagentes por meio de habilidades, comandos de barra e *hooks*. O sistema incorpora cinco mecanismos de governança: integridade de memória (M1), validação cruzada (M2), resistência a injeção via *framework* estruturado de análise (M3), monitoramento comportamental (M4) e quarentena (M5). A avaliação é conduzida em um laboratório instrumentado com Sysmon e Elastic Security sobre um domínio Windows, comparando o MAS-Hunt completo (C1-full) a três condições de referência (C1-nohardening, C2-naive e C3-Elastic) quanto a desempenho de detecção, resiliência adversarial e taxa de falsos positivos. Na matriz v2 com 1.092 janelas adversariais ricas em sinal, todas as três condições baseadas em modelo de linguagem superaram a linha de base SIEM C3-Elastic ($F2 = 0,150$) por aproximadamente 4,5 vezes, ganho dirigido pela revocação. A governança multiagente, porém, não melhorou o F2 de detecção sobre o modelo de passagem única: C1-full (0,675) e C2-naive (0,672) mostraram-se estatisticamente indistinguíveis, com intervalos de confiança sobrepostos. O valor distintivo da governança manifestou-se na resistência à injeção adversarial: somente C1-full detectou e sinalizou conteúdo adversarial injetado, marcando 407 janelas contaminadas contra zero nas demais condições. Conclui-se que o analista agêntico supera dramaticamente as regras estáticas de SIEM, e que a governança multiagente agrega resistência à injeção e auditabilidade — e não maior acurácia de detecção — sobre o modelo de linguagem isolado.

Palavras-chave: caça a ameaças; sistemas multiagente; segurança de agentes de IA; injeção adversarial; governança; Elastic Stack; Windows.

Abstract

The escalating sophistication and evasive nature of contemporary cyber threats — characterized by living-off-the-land (LOLBin) techniques, prolonged attacker dwell time, and the cognitive overload imposed on security analysts — expose the limits of reactive, signature-based architectures. Autonomous agents powered by large language models (LLMs) offer a proactive alternative, yet they are themselves vulnerable to adversarial manipulation, including memory poisoning, prompt injection, and malfunction amplification. This dissertation presents MAS-Hunt (Multi-Agent System for Threat Hunting), a security-first three-layer multi-agent architecture — Governance Board, Managers, and Workers/Hunters — implemented as a Claude Code plugin that orchestrates subagents through skills, slash commands, and hooks. The system embeds five governance mechanisms: memory integrity (M1), cross-validation (M2), injection resistance via a structured analysis framework (M3), behavioral monitoring (M4), and quarantine (M5). The evaluation is conducted in a laboratory instrumented with Sysmon and Elastic Security over a Windows domain, comparing the full MAS-Hunt (C1-full) against three baselines (C1-nohardening, C2-naive, and C3-Elastic) with respect to detection performance, adversarial resilience, and false-positive rate. In the v2 matrix with 1,092 signal-rich adversarial windows, all three language-model conditions outperformed the C3-Elastic SIEM baseline ($F2 = 0.150$) by roughly 4.5 times, a gain driven by recall. Multi-agent governance, however, did not improve detection F2 over the single-pass model: C1-full (0.675) and C2-naive (0.672) were statistically indistinguishable, with overlapping confidence intervals. The distinctive value of governance emerged in adversarial-injection resistance: only C1-full detected and flagged injected adversarial content, marking 407 contaminated windows versus zero for the other conditions. We conclude that the agentic analyst dramatically outperforms static SIEM rules, and that multi-agent governance adds injection resistance and auditability — rather than higher detection accuracy — over the standalone language model.

Keywords: threat hunting; multi-agent systems; AI agent security; adversarial injection; governance; Elastic Stack; Windows.

Sumário

1	INTRODUÇÃO	8
1.1	Contextualização	8
1.2	Definição do Problema	9
1.3	Lacuna de Pesquisa	10
1.4	Objetivos	11
1.4.1	Objetivo Geral	11
1.4.2	Objetivos Específicos	11
1.5	Hipótese	11
1.6	Contribuições	12
1.7	Organização do Texto	12
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	Caça a Ameaças e Detecção Comportamental	14
2.1.1	Limitações das defesas convencionais	14
2.1.2	O ciclo da caça dirigida por hipóteses	15
2.1.3	A falácia da taxa-base e seus efeitos sobre o falso positivo	15
2.2	Grandes Modelos de Linguagem e Agentes de IA	16
2.2.1	Modelos de linguagem como mecanismo de raciocínio	16
2.2.2	De modelos a agentes	16
2.3	Sistemas Multiagente	17
2.3.1	Topologias de orquestração e robustez	17
2.4	Segurança de Agentes de IA	18
2.4.1	Envenenamento de memória e de bases de conhecimento	18
2.4.2	Injeção de prompt direta e indireta	18
2.4.3	Amplificação de mau funcionamento e jailbreaks infecciosos	19
2.4.4	Governança e conformidade de sistemas de IA	19
2.5	Elastic Security, Sysmon e Telemetria Windows	20
2.5.1	O Elastic Stack como terreno de caça	20
2.5.2	Sysmon e os identificadores de evento	20
2.5.3	A lacuna de telemetria do EID 1 e o <i>health-gate</i>	21
2.6	Técnicas LOLBin e o MITRE ATT&CK	21
2.6.1	O framework MITRE ATT&CK	21
2.6.2	Binários do tipo Living-off-the-Land	22
2.7	Injeção Adversarial de Logs	22
2.8	Síntese do Capítulo	23
3	TRABALHOS RELACIONADOS	24

3.1	Caça a Ameaças e Detecção Comportamental Assistidas por IA . . .	24
3.2	Sistemas Multiagente para Segurança e sua Governança	25
3.3	Ataques Adversariais a Agentes Baseados em LLM	26
3.4	Automação de Detecção em Ambientes SIEM/SOC	27
3.5	Síntese Comparativa	28
3.6	A Lacuna que o MAS-Hunt Preenche	28
4	ARQUITETURA MAS-HUNT	31
4.1	Visão Geral da Arquitetura em Três Camadas	31
4.2	Papéis e Responsabilidades dos Agentes	32
4.2.1	Camada 1 — Conselho de Governança	32
4.2.2	Camada 2 — Gerentes	34
4.2.3	Camada 3 — Caçadores	35
4.3	Os Mecanismos de Governança M1–M5	35
4.3.1	M1 — Integridade de Memória	36
4.3.2	M2 — Validação Cruzada	36
4.3.3	M3 — Resistência a Injeção	36
4.3.4	M4 — Monitoramento Comportamental	36
4.3.5	M5 — Quarentena	37
4.3.6	Síntese: a defesa em profundidade dos agentes	37
4.4	O Framework M3 de Análise Estruturada	38
4.5	Implementação como Plugin do Claude Code	38
4.5.0.0.1	Habilidades (<i>skills</i>).	39
4.5.0.0.2	Comandos de barra (<i>slash commands</i>).	39
4.5.0.0.3	<i>Hooks</i>	39
4.6	Protocolos de Mensagens Tipadas e Governança	40
4.6.1	Por que a Orquestração Real Difere de uma Chamada LLM Isolada	41
5	METODOLOGIA EXPERIMENTAL	43
5.1	Ambiente de Laboratório Instrumentado	43
5.1.1	Topologia de domínio Active Directory	43
5.1.2	Instrumentação de telemetria	44
5.1.3	Plataforma analítica: Elastic 9.3.4	44
5.2	Corpus de Ataque e Verdade de Referência	45
5.2.1	Os 185 <i>playbooks</i> em nove famílias	45
5.2.2	Crítério de verdade de referência (<i>ground truth</i>)	46
5.3	Desenho Adversarial de Injeção de Logs	46
5.3.1	Quatro modos de injeção	47
5.3.2	Três níveis de sofisticação	47
5.3.3	Estrutura fatorial e <i>payloads</i>	47
5.4	Condições Comparadas	48

5.5	<i>Pipeline de Medição</i>	49
5.5.1	Janelas por <i>playbook</i> e extração de telemetria	49
5.5.2	Extração <i>prepare-v2</i> : priorização de EID1	49
5.5.3	O gatilho de saúde de telemetria (<i>health-gate</i>)	50
5.5.3.0.1	Por que o gatilho é necessário.	51
5.5.4	Reprodutibilidade: restauração limpa e <i>snapshots</i>	51
5.6	Métricas e Análise Estatística	52
5.6.1	Métricas de detecção	52
5.6.2	Métricas de resiliência adversarial	53
5.6.3	Análise estatística por <i>bootstrap</i>	53
5.7	Limitações e Ameaças à Validade	54
5.7.1	A lacuna de telemetria de EID1	54
5.7.2	Outras ameaças à validade	55
5.7.2.0.1	Validade interna.	55
5.7.2.0.2	Validade externa.	55
5.7.2.0.3	Validade de construto.	55
5.7.2.0.4	Estabilidade do laboratório.	55
6	RESULTADOS	57
6.1	Conjunto de Dados Analisado e Recorte Inferencial	57
6.2	Desempenho de Detecção: Matriz das Quatro Condições	57
6.2.1	Modelo de linguagem versus regras estáticas: o contraste dominante	58
6.2.2	Orquestração multiagente versus passagem única: ausência de ganho de F2	59
6.3	Desempenho por Nível de Sofisticação da Injeção	59
6.4	Taxa de Falsos Positivos	60
6.5	Resistência à Injeção Adversarial	60
6.5.1	Efeito da ablação dos mecanismos de endurecimento	61
6.6	Resiliência sob Ataque Reforçado e Controle de Vazamento de Rótulo	62
6.6.0.0.1	Desenho.	62
6.6.1	O ataque não consegue “lavar” a atividade maliciosa	63
6.6.2	O ganho de auditabilidade é robusto entre níveis de modelo	63
6.6.3	Indício sugestivo, não significativo, de auxílio à detecção do modelo fraco	64
6.7	Síntese Quantitativa	64
7	DISCUSSÃO	66
7.1	Interpretação dos Três Achados	66
7.1.1	O analista de modelo de linguagem como ruptura de patamar	66
7.1.2	A governança não melhora o F2 de detecção — relato honesto	67
7.1.3	A resistência à injeção como valor distintivo	67
7.2	Confronto com a Hipótese e Reformulação da Contribuição	68
7.3	Implicações Práticas e de Pesquisa	69

7.3.1	Para operações de SOC	69
7.3.2	Para a avaliação de agentes de segurança	69
7.4	Limitações	69
7.4.0.0.1	A lacuna estrutural de telemetria de EID1.	69
7.4.0.0.2	Granularidade da matriz.	70
7.4.0.0.3	Taxa de falsos positivos por janela benigna.	70
7.4.0.0.4	Vazamento de rótulo na telemetria do laboratório v2.	70
7.5	Ameaças à Validade	70
7.5.0.0.1	Validade interna.	70
7.5.0.0.2	Validade de construto.	71
7.5.0.0.3	Validade externa.	71
8	CONCLUSÃO	72
8.1	Síntese do Trabalho	72
8.2	Resposta à Questão de Pesquisa	72
8.3	Contribuições (Reformuladas)	73
8.4	Trabalhos Futuros	73
8.5	Considerações Finais	74
	REFERÊNCIAS	75

1 Introdução

1.1 Contextualização

O cenário contemporâneo da segurança cibernética é definido pela velocidade e pela sofisticação crescentes das operações adversárias. Ataques de *ransomware*, campanhas de *phishing* e a exploração de vulnerabilidades tornaram-se mais frequentes e complexos, obrigando as organizações a repensar continuamente suas estratégias de defesa [Valencia 2024]. A proliferação de fornecedores comerciais de vigilância e de mercados clandestinos de credenciais válidas amplia o acesso a capacidades ofensivas avançadas, de modo que atacantes alcançam, com regularidade, os ambientes corporativos antes que qualquer alerta seja gerado.

Nesse contexto, a busca proativa por ameaças, ou *threat hunting*, consolidou-se como uma prática que transcende a reatividade dos sistemas tradicionais. Em vez de aguardar que um alerta seja disparado, os profissionais de segurança adotam técnicas de investigação forense para examinar redes, sistemas e estações de trabalho à procura de sinais de comprometimento [Sindiramutty 2024]. Essa abordagem ativa permite identificar e neutralizar indicadores de atividade maliciosa antes que sejam plenamente explorados pelos atacantes, reduzindo a janela de oportunidade do adversário. A literatura descreve essa inversão de postura como a transformação do alvo em caçador, com ênfase no ciclo de vida da caça e no papel emergente da inteligência artificial [Hillier e Karroubi 2022].

Um traço distintivo das ameaças modernas é a adoção sistemática de técnicas de *living-off-the-land* (LOLBins), nas quais o atacante abusa de binários, *scripts* e utilitários legítimos já presentes no sistema operacional — a exemplo de **PowerShell**, **wmic**, **certutil**, **rundll32** e **mshta** — para executar suas ações. Por se valerem de ferramentas confiáveis e assinadas pelo próprio fornecedor do sistema, essas técnicas produzem telemetria que, isoladamente, é praticamente indistinguível da atividade administrativa benigna. O resultado é uma evasão eficaz das defesas baseadas em assinaturas e listas de bloqueio, deslocando o ônus da detecção para a análise comportamental e contextual dos eventos. Ambientes Windows, predominantes em redes corporativas e sustentados por uma rica superfície de utilitários nativos, são especialmente suscetíveis a esse padrão de abuso.

A esse desafio técnico soma-se um desafio humano e operacional. As soluções de detecção contemporâneas, mesmo quando dotadas de modelos estatísticos acurados, estão sujeitas à falácia da taxa-base: em ambientes nos quais a atividade maliciosa é rara, mesmo classificadores de elevada acurácia geram um volume desproporcional de falsos positivos, tornando-se operacionalmente ineficazes. O analista de segurança, por sua vez, é submetido a uma sobrecarga cognitiva permanente, dividido entre o triagem de alertas de baixa qualidade e a investigação aprofundada de hipóteses relevantes. Essa sobrecarga compromete a capacidade de resposta da organização e favorece a fadiga de alertas, na

qual eventos genuinamente críticos podem ser negligenciados em meio ao ruído.

Os agentes de inteligência artificial apresentam-se como uma resposta promissora a esse cenário. Tais agentes são estruturas de *software* que ampliam as capacidades dos grandes modelos de linguagem (LLMs — *Large Language Models*), permitindo que estes raciocinem sobre evidências, interajam com ferramentas externas e executem ações no sistema [Valencia 2024]. O mecanismo de atenção introduzido por Vaswani et al. [Vaswani et al. 2017] pavimentou o caminho para esses modelos, e trabalhos subsequentes demonstraram seu potencial de coordenação e tomada de decisão conjunta em arranjos multiagente [Agashe et al. 2025]. Na segurança cibernética, propostas como o HuntGPT [Ali e Kostakos 2023] e modelos de geração de hipóteses para a caça a ameaças [Yi e Kim 2024] ilustram a viabilidade de aplicar LLMs à detecção proativa, conjugando explicabilidade e raciocínio investigativo.

Essa autonomia, contudo, é uma faca de dois gumes. A mesma flexibilidade que torna os agentes poderosos os expõe a uma classe de ataques que tem por alvo o próprio agente, e não apenas o sistema defendido. Pesquisas recentes documentam o envenenamento de memória e de bases de recuperação aumentada (RAG), a injeção indireta de *prompt* dissimulada em conteúdo externo e a amplificação de mau funcionamento, na qual um erro inicial desencadeia um ciclo de ações inúteis que degrada o desempenho do agente. Demonstrou-se, ainda, a possibilidade de *jailbreaks* infecciosos, em que uma única entrada adversarial se propaga por uma população de agentes. Diante disso, qualquer arquitetura defensiva que não modele um adversário igualmente capaz de atacar a camada de raciocínio é, por construção, incompleta.

A governança em segurança cibernética emerge, portanto, como pilar para alinhar as estratégias de defesa às necessidades do negócio e para coordenar, de forma segura, o trabalho dos agentes [Oesch et al. 2025]. Frameworks consolidados — como COBIT, NIST CSF, ISO 27001 e o recente ISO 42001 — precisam evoluir para incorporar os riscos específicos introduzidos pela integração de tecnologias de IA [McIntosh et al. 2024, ISACA 2025]. A presente dissertação situa-se nessa interseção: propõe a governança multiagente não como camada burocrática acessória, mas como mecanismo técnico de resiliência, capaz de conter a propagação de comprometimentos e de validar achados antes que se convertam em decisões.

1.2 Definição do Problema

O problema de pesquisa pode ser enunciado pela tensão entre duas exigências aparentemente conflitantes. De um lado, demanda-se um sistema de caça a ameaças *eficaz*, capaz de operar sobre a telemetria viva de um ambiente corporativo Windows, detectar técnicas evasivas de *living-off-the-land* e reduzir o volume de falsos positivos que sobrecarrega os analistas. De outro, exige-se que esse mesmo sistema seja *resiliente*: como os agentes que o compõem são suscetíveis a envenenamento de memória, injeção de *prompt* e amplificação

de mau funcionamento, a automação introduz uma nova superfície de ataque que, se explorada, pode subverter justamente a função de defesa que o sistema deveria exercer.

Formula-se, assim, a questão central de pesquisa: *como projetar uma arquitetura multiagente para caça a ameaças que opere de forma eficaz sobre a telemetria corporativa real e, simultaneamente, demonstre resiliência diante de ataques adversariais dirigidos aos próprios agentes?* A resposta a essa questão não se esgota na escolha de um modelo de linguagem mais capaz; ela depende, sobretudo, da arquitetura de coordenação e dos mecanismos de governança que regulam o fluxo de informação e a tomada de decisão entre os agentes.

1.3 Lacuna de Pesquisa

A revisão da literatura revela que, embora os agentes de IA tenham avançado de tarefas supervisionadas e específicas para sistemas de raciocínio autônomo, as arquiteturas atuais apresentam lacunas críticas quando confrontadas com um adversário inteligente. Identificam-se, em particular, três lacunas que esta dissertação se propõe a endereçar:

1. **Ausência de orquestração multiagente robusta.** Faltam *frameworks* que impeçam que o comprometimento de um único agente se propague sistemicamente, contendo a falha em vez de amplificá-la. A maioria das propostas concentra-se no desempenho de detecção e negligencia a contenção de comprometimentos internos.
2. **Ausência de validação de integridade em tempo de execução.** Os mecanismos de memória e as bases de conhecimento dos agentes carecem de validação contínua contra envenenamento furtivo de dados, deixando o raciocínio do agente vulnerável a manipulações que persistem entre interações.
3. **Autocorreção ineficaz frente à degradação comportamental.** Os agentes não dispõem de mecanismos confiáveis para detectar e reverter sua própria degradação, permanecendo expostos à amplificação de mau funcionamento e a ataques de negação de serviço sobre o ciclo de raciocínio.

Soma-se a essas lacunas a escassez de avaliações conduzidas em condições realistas — sobre telemetria viva, com instrumentação de produção e contra adversários que atacam a camada de agentes — e a ausência de uma comparação sistemática entre arquiteturas governadas e linhas de base sem governança. É precisamente nessa interseção entre resiliência adversarial, validação de integridade e operação realista que o MAS-Hunt se posiciona.

1.4 Objetivos

1.4.1 Objetivo Geral

Projetar, implementar e avaliar o MAS-Hunt, uma arquitetura de governança multiagente para a busca proativa por ameaças cibernéticas em ambientes Windows, capaz de operar sobre telemetria viva e de demonstrar resiliência mensurável a ataques adversariais dirigidos aos agentes, em comparação a linhas de base sem governança.

1.4.2 Objetivos Específicos

Para alcançar o objetivo geral, estabelecem-se os seguintes objetivos específicos:

1. Revisar criticamente a literatura sobre caça a ameaças e detecção comportamental, agentes baseados em LLMs e sistemas multiagente, segurança de agentes (envenenamento de memória, injeção de *prompt* e amplificação de mau funcionamento) e técnicas de *living-off-the-land* mapeadas ao MITRE ATT&CK.
2. Especificar a arquitetura MAS-Hunt em três camadas — Conselho de Governança, Gerentes e Trabalhadores/Caçadores — detalhando os papéis dos agentes, os protocolos de mensagens tipadas e os cinco mecanismos de governança M1 (integridade de memória), M2 (validação cruzada), M3 (resistência a injeção), M4 (monitoramento comportamental) e M5 (quarentena).
3. Implementar o MAS-Hunt como um *plugin* do Claude Code, materializado por meio de habilidades (*skills*), comandos de barra (*slash commands*) e *hooks* que orquestram subagentes pelas capacidades nativas do agente principal.
4. Construir um laboratório de avaliação instrumentado com Sysmon e Elastic Security sobre um domínio Windows, com um corpus de *playbooks* de técnicas LOLBin (malicioso versus benigno) e um desenho adversarial que combina modos de injeção e níveis de severidade.
5. Avaliar empiricamente o MAS-Hunt completo (C1-full) frente às condições de referência (C1-nohardening, C2-naive e C3-Elastic) quanto ao desempenho de detecção, à resiliência adversarial e à taxa de falsos positivos, com análise estatística por intervalos de confiança via *bootstrap*.

1.5 Hipótese

A hipótese central desta dissertação é a seguinte: *a arquitetura MAS-Hunt, ao incorporar mecanismos de governança multiagente (M1–M5), apresenta uma redução estatisticamente significativa da suscetibilidade a ataques de envenenamento de memória e de amplificação de mau funcionamento — medida pela taxa de sucesso do ataque (Attack Success Rate)*

— em comparação a uma linha de base multiagente sem endurecimento e a um agente de passagem única, preservando ou aprimorando o desempenho de detecção e reduzindo a taxa de falsos positivos em relação às regras de detecção padrão do Elastic Security.

Dessa hipótese central derivam-se duas hipóteses operacionais subordinadas: (i) a governança contribui de forma marginal e mensurável para a resiliência, de modo que a ablação dos mecanismos de endurecimento (C1-nohardening) eleva a taxa de sucesso do ataque em relação ao sistema completo (C1-full); e (ii) a orquestração multiagente governada reduz a taxa de falsos positivos em comparação tanto ao agente de passagem única (C2-naive) quanto às regras padrão do Elastic (C3-Elastic), sem prejuízo da revocação.

1.6 Contribuições

As principais contribuições desta dissertação são:

1. **Uma síntese crítica** do panorama atual de ameaças e do estado da arte em vulnerabilidades de agentes de IA, articulando os riscos da camada de agentes (envenenamento de memória, injeção de *prompt* e amplificação de mau funcionamento) às exigências operacionais da caça a ameaças em ambientes Windows.
2. **A arquitetura MAS-Hunt**, uma proposta de orquestração multiagente em três camadas com cinco mecanismos de governança (M1–M5) e um *framework* estruturado de análise (OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR) que confere resistência a injeção adversarial.
3. **Uma implementação concreta** do MAS-Hunt como *plugin* do Claude Code — composto por habilidades, comandos de barra e *hooks* que orquestram subagentes —, demonstrando que a governança multiagente pode ser realizada sobre capacidades nativas de orquestração do agente principal, e não apenas por meio de *scripts* isolados.
4. **Um arcabouço de avaliação reprodutível**, incluindo o laboratório instrumentado, o corpus de *playbooks* LOLBin, o desenho adversarial e o pipeline de medição com *health-gate* de telemetria, que permite comparar arquiteturas governadas e não governadas sob métricas e análise estatística bem definidas.

1.7 Organização do Texto

O restante desta dissertação está organizado em sete capítulos. O Capítulo 1 — este capítulo — apresentou o contexto, o problema, a lacuna de pesquisa, os objetivos, a hipótese e as contribuições do trabalho. O Capítulo 2 desenvolve a fundamentação teórica, abrangendo a caça a ameaças e a detecção comportamental, os LLMs e agentes, os sistemas multiagente, a segurança de agentes, a telemetria Windows com Sysmon e Elastic Security, as técnicas de *living-off-the-land* mapeadas ao MITRE ATT&CK e a injeção adversarial de

logs. O Capítulo 3 examina os trabalhos relacionados, comparando abordagens existentes e situando as lacunas que o MAS-Hunt se propõe a preencher.

O Capítulo 4 detalha a arquitetura MAS-Hunt: a visão geral das três camadas, os papéis dos agentes, os mecanismos M1–M5, o *framework* M3 de análise, a implementação como *plugin* do Claude Code e os protocolos de mensagens tipadas e de governança. O Capítulo 5 descreve a metodologia experimental, incluindo o laboratório, o corpus de *playbooks*, o desenho adversarial, as quatro condições experimentais, o pipeline de medição e a análise estatística, documentando honestamente as limitações de telemetria e o papel do *health-gate* na sua mitigação.

O Capítulo 6 apresenta os resultados obtidos; o Capítulo 7 discute suas implicações, limitações e ameaças à validade; e o Capítulo 8 conclui o trabalho, recapitulando as contribuições e apontando direções para pesquisas futuras.

2 Fundamentação Teórica

Este capítulo estabelece o arcabouço conceitual sobre o qual o MAS-Hunt (*Multi-Agent System for Threat Hunting*) é construído e avaliado. A exposição parte da prática de caça a ameaças e da detecção comportamental, percorre os fundamentos dos grandes modelos de linguagem e dos agentes de inteligência artificial, examina os sistemas multiagente e — central para a contribuição desta dissertação — a segurança dos próprios agentes, antes de detalhar a infraestrutura de telemetria do ecossistema Windows/Elastic, as técnicas de abuso de binários nativos (*Living-off-the-Land Binaries*, LOLBin) catalogadas pelo MITRE ATT&CK e, por fim, a injeção adversarial de logs. Sempre que pertinente, os conceitos são amarrados aos cinco mecanismos de governança do MAS-Hunt — M1 (Integridade de Memória), M2 (Validação Cruzada), M3 (Resistência a Injeção), M4 (Monitoramento Comportamental) e M5 (Quarentena) — e às quatro condições experimentais (C1-full, C1-nohardening, C2-naive e C3-Elastic), de modo que a metodologia do Capítulo 5 e a discussão dos resultados encontrem aqui o seu embasamento.

2.1 Caça a Ameaças e Detecção Comportamental

A caça a ameaças (*threat hunting*) consolidou-se como uma mudança de paradigma na defesa cibernética, deslocando o eixo da postura reativa — que aguarda o disparo de um alerta — para uma busca ativa e iterativa por indícios de comprometimento que tenham escapado dos controles automatizados [Sindiramutty 2024]. A premissa operacional é assumir a violação (*assume breach*): parte-se do princípio de que um adversário pode já estar presente no ambiente e, por conseguinte, o objetivo passa a ser reduzir o tempo de permanência do atacante (*dwell time*) e mitigar o dano antes que os objetivos da intrusão sejam alcançados [Hillier e Karroubi 2022]. Essa urgência é quantificada por relatórios setoriais: o tempo mediano global de permanência do atacante mantém-se na ordem de poucos dias a semanas, intervalo suficiente para movimentação lateral e exfiltração de dados [Mandiant 2025], ao passo que violações envolvendo terceiros respondem por parcela expressiva dos incidentes registrados [Verizon Business 2025].

2.1.1 Limitações das defesas convencionais

As arquiteturas tradicionais, ancoradas em detecção por assinatura, são inerentemente reativas: mostram-se eficazes contra ameaças conhecidas, mas incapazes de identificar artefatos novos, polimórficos ou de dia zero, para os quais nenhuma assinatura existe [Ucci, Aniello e Baldoni 2019]. A resposta histórica a essa limitação foi a análise comportamental, conduzida por sistemas de detecção de intrusão baseados em host (HIDS) ou em rede (NIDS). Tais sistemas, contudo, operam frequentemente de forma isolada: um

HIDS pode observar um processo suspeito e um NIDS pode sinalizar uma conexão de rede anômala, mas, sem mecanismos de correlação, falham em conectar eventos dispersos na narrativa coerente de um ataque coordenado. A caça a ameaças surge justamente para suprir essa lacuna de correlação por meio da investigação dirigida por hipóteses [Yi e Kim 2024].

2.1.2 O ciclo da caça dirigida por hipóteses

A modalidade mais madura de caça a ameaças é a dirigida por hipóteses, na qual o analista formula uma conjectura sobre comportamento adversário plausível — por exemplo, “um atacante está utilizando `certutil.exe` para baixar um estágio secundário” — e então interroga a telemetria disponível em busca de evidências que a confirmem ou refutem [Yi e Kim 2024]. Esse ciclo articula-se em etapas de observação, formulação de hipótese, coleta de evidências e classificação. A automação dessas etapas por meio de aprendizado de máquina e de modelos de linguagem é objeto de pesquisa crescente, em trabalhos que vão da análise de logs [A et al. 2024] à detecção de malware em dispositivos de Internet das Coisas mediante redes neurais recorrentes [HaddadPajouh et al. 2018], passando por surveys dedicados à aplicação da caça a ameaças contra ransomware [Aldauji, Batarfi e Bayousef 2022] e por abordagens de aprendizado federado para detecção de ameaças persistentes avançadas (APT) [Thi et al. 2022]. Sistemas como o HuntGPT demonstraram a viabilidade de acoplar detecção de anomalias por aprendizado de máquina a modelos de linguagem dotados de IA explicável, oferecendo ao analista justificativas em linguagem natural [Ali e Kostakos 2023]; já propostas como o TIRA exploram a automação do ciclo completo de caça, detecção e perfilamento de atores [Yulianto e Ngo 2024], e estudos sobre a autonomia da defesa cibernética delineiam o horizonte de longo prazo dessa transição [Oesch et al. 2025]. A semente conceitual do framework M3 do MAS-Hunt — a cadeia OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR — é uma formalização operacional desse ciclo, projetada de modo a ser, simultaneamente, um protocolo de raciocínio e uma barreira de resistência a manipulação adversarial, conforme detalhado na Seção 2.4.

2.1.3 A falácia da taxa-base e seus efeitos sobre o falso positivo

Um obstáculo estatístico fundamental atravessa qualquer sistema de detecção que busque eventos raros: a falácia da taxa-base (*base rate fallacy*). Mesmo um classificador de elevada acurácia produz um volume avassalador de falsos positivos quando o fenômeno-alvo é raro na população observada [Dolan-Gavitt 2025]. Formalmente, pelo teorema de Bayes, a probabilidade *a posteriori* de que um alerta corresponda a uma ameaça real é

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}, \quad (2.1)$$

em que $P(A)$ é a prevalência da ameaça, $P(B | A)$ a sensibilidade do detector e $P(B)$ a probabilidade total de um alerta. Quando $P(A)$ é muito pequeno — o que é típico em telemetria corporativa, na qual a esmagadora maioria dos eventos é benigna — o termo de falsos positivos no denominador domina, e o valor preditivo positivo desaba, ainda que sensibilidade e especificidade sejam altas. Essa realidade tem duas consequências de projeto para o MAS-Hunt. Primeiro, justifica a escolha da métrica F2 (Seção 2.6.2) e a comparação explícita da Taxa de Falsos Positivos (FPR) contra as regras padrão do Elastic Security (condição C3-Elastic). Segundo, motiva a validação determinística: em vez de confiar na saída estocástica do modelo, exige-se evidência verificável — por exemplo, o acesso comprovado a um artefato-isca (*canary*) de identificador único e difícil de adivinhar — antes de elevar a confiança em um achado [Dolan-Gavitt 2025], desacoplando a descoberta flexível, conduzida pela IA, da validação rígida e confiável, conduzida por código não-IA.

2.2 Grandes Modelos de Linguagem e Agentes de IA

2.2.1 Modelos de linguagem como mecanismo de raciocínio

Os grandes modelos de linguagem (*Large Language Models*, LLMs) são redes neurais de larga escala fundamentadas na arquitetura *Transformer*, cujo mecanismo de autoatenção permite modelar dependências de longo alcance em sequências de texto [Vaswani et al. 2017]. Treinados sobre vastos corpora, exibem capacidades emergentes de compreensão, síntese e raciocínio em linguagem natural que viabilizam, sem treinamento específico de tarefa, a interpretação de telemetria de segurança, a formulação de hipóteses e a geração de regras de detecção. Tais capacidades vêm sendo ampliadas por técnicas de computação em tempo de inferência, como o raciocínio latente recorrente, que aprofundam o processo deliberativo do modelo sem aumentar proporcionalmente o número de parâmetros [Geiping et al. 2025]. Dois atributos dos LLMs são, entretanto, decisivos para o projeto de um sistema de defesa: sua saída é, por construção, estocástica — o que torna indispensável um mecanismo de validação e realimentação por especialistas [Sindiramutty 2024] — e seu comportamento é integralmente condicionado pelo contexto textual fornecido, propriedade que abre a superfície de ataque por injeção de instruções discutida na Seção 2.4.

2.2.2 De modelos a agentes

Um agente de inteligência artificial é uma estrutura de software que estende um LLM com a capacidade de perceber um ambiente, raciocinar sobre objetivos e executar ações por meio de ferramentas — consultas a bancos de dados, chamadas a APIs, execução de scripts — em um laço iterativo de percepção, decisão e atuação [Valencia 2024, Guo et al. 2024]. É essa autonomia instrumental que distingue um agente de um simples *chatbot*: o agente

não apenas responde, mas age sobre o sistema. No MAS-Hunt, os agentes executores interagem diretamente com o Elastic Stack, formulando e refinando hipóteses, consultando o Elasticsearch em busca de atividade anômala, criando regras de detecção pela API do Elastic Security e enriquecendo achados com inteligência externa. A mesma autonomia que confere poder ao agente, contudo, é também a origem de sua fragilidade: a literatura recente demonstra que agentes autônomos podem ser explorados de maneiras que não se aplicam a modelos passivos [Valencia 2024, Chen et al. 2024], tema central da Seção 2.4. Vale ainda registrar a natureza de uso dual dessas capacidades: os mesmos princípios agênticos que sustentam a defesa habilitam a ofensiva, como evidenciado por sistemas de teste de intrusão autônomos baseados em aprendizado por reforço [Moreno et al. 2025] e por agentes ofensivos de segurança [Valencia 2024]. Qualquer arquitetura defensiva que não modele um adversário igualmente capaz é, por isso, fundamentalmente incompleta.

2.3 Sistemas Multiagente

Um sistema multiagente (*Multi-Agent System*, MAS) é composto por múltiplos agentes autônomos que interagem entre si e com o ambiente para alcançar objetivos individuais ou coletivos. Esse paradigma é particularmente adequado à segurança cibernética, domínio em que a defesa requer a coordenação de sensores distribuídos e de componentes de decisão. A coordenação entre agentes é um problema clássico da inteligência artificial distribuída, estudado sob a ótica de organizações, planos e escalonamento [Durfee 1993], e revigorado pelos LLMs, que passaram a atuar como mecanismo de raciocínio dos agentes [Agashe et al. 2025]. A pesquisa contemporânea explora desde a propagação de intenções e raciocínio para coordenação [Qiu et al. 2024], a cooperação proativa [Zhang et al. 2024] e o aprendizado de coordenação *zero-shot* [Li et al. 2023], até arquiteturas hierárquicas de agentes de linguagem para coordenação humano-IA em tempo real [Liu et al. 2024], a redução de computação redundante por execução localmente centralizada [Bai e Sugawara 2024] e o uso de modelos de fundação multimodais para coordenação distribuída [Mahmud, Goldfajn e Zilberstein 2025].

2.3.1 Topologias de orquestração e robustez

A escolha da topologia de orquestração tem efeito direto sobre a robustez do sistema. Arquiteturas hierárquicas — nas quais uma camada superior coordena camadas subordinadas — exibem a menor degradação de desempenho sob condições adversariais quando comparadas a topologias plenamente distribuídas ou a redes *peer-to-peer* [Huang et al. 2025]. Esse achado fundamenta diretamente a arquitetura de três camadas do MAS-Hunt — **Conselho de Governança** (*Board*) → **Gerentes** (*Managers*) → **Trabalhadores/Caçadores** (*Workers/Hunters*). Abordagens correlatas reforçam a importância do projeto estrutural: a geração de robustez por adversários auxiliares evolutivos demonstra como exposições controladas a ataques podem endurecer a coordenação [Yuan et al. 2023], ao passo que

protocolos de tolerância bizantina, inspirados em mecanismos de consenso de cadeia de blocos, buscam impedir que um único agente comprometido subverta a decisão coletiva [Chen et al. 2024]. O Conselho de Governança do MAS-Hunt incorpora esse princípio por meio de um mecanismo de consenso deliberativo entre múltiplos agentes, no qual a aprovação de uma diretriz estratégica exige supermaioria, elevando a barreira para que uma falha ou comprometimento isolado altere o rumo da operação.

2.4 Segurança de Agentes de IA

A integração de agentes autônomos à segurança cibernética introduz uma classe de riscos qualitativamente distinta: o próprio defensor torna-se alvo. Esta seção sistematiza as três famílias de ataque que o MAS-Hunt foi projetado para resistir e que motivam os mecanismos M1 a M5. Trata-se da contribuição conceitual nuclear desta dissertação, pois desloca a discussão do desempenho de detecção para a resiliência do agente.

2.4.1 Envenenamento de memória e de bases de conhecimento

Os componentes cognitivos do agente — sua memória de longo prazo e as bases de conhecimento consultadas por geração aumentada por recuperação (*Retrieval-Augmented Generation*, RAG) — constituem alvo privilegiado. Demonstrou-se que uma base pode ser envenenada com uma taxa de contaminação inferior a 0,1% e ainda assim induzir comportamento malicioso direcionado com taxa de sucesso de ataque superior a 80%, forçando o agente a recuperar e a agir sobre conteúdo adulterado [Chen et al. 2024]. A insidiosidade do ataque reside em sua furtividade: a contaminação é estatisticamente diluída e dificilmente detectável por inspeção superficial. O MAS-Hunt responde com o mecanismo **M1 (Integridade de Memória)**, que submete os resultados de recuperação a validação multifonte e a protocolos de sanitização, e com o **M5 (Quarentena)**, que isola dados anômalos — por exemplo, registros com carimbos de tempo fabricados — antes que contaminem o raciocínio subsequente.

2.4.2 Injeção de prompt direta e indireta

A injeção de prompt explora o fato de que um LLM não distingue, no nível do texto, entre instruções legítimas do operador e instruções incorporadas aos dados que processa. Na variante indireta, o atacante esconde diretivas maliciosas em conteúdo externo que o agente virá a ler — uma página web, um documento ou, no cenário desta dissertação, um registro de log — de modo que a simples leitura desse conteúdo redirecione o comportamento do agente. Em agentes integrados a ferramentas, esse vetor é especialmente perigoso, pois a instrução injetada pode disparar ações reais sobre o sistema; o benchmark InjecAgent quantifica precisamente essa exposição em agentes que operam com ferramentas [Zhan et al. 2024]. O MAS-Hunt confronta esse vetor com o mecanismo

M3 (Resistência a Injeção), operacionalizado pelo framework de análise OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR, cuja disciplina exige que a classificação derive exclusivamente de observáveis verificáveis da telemetria, sinalizando — e não obedecendo — qualquer conteúdo que se apresente como instrução. É justamente a eficácia de M3 que o desenho experimental adversarial — quatro modos de injeção combinados a três níveis de furtividade (*overt*, *encoded*, *steganographic*) — pretende mensurar por meio do *Attack Success Rate* (ASR) e de sua variante não-atribuída (NASR).

2.4.3 Amplificação de mau funcionamento e jailbreaks infecciosos

A terceira família ataca não a memória nem a entrada, mas o próprio processo de raciocínio. Na amplificação de mau funcionamento (*malfunction amplification*), um erro menor é deliberadamente induzido e propaga-se em um laço de realimentação de ações inúteis, degradando o agente a ponto de configurar uma negação de serviço; taxas de falha superiores a 80% foram reportadas [Zhang et al. 2025]. Em sistemas multiagente, o risco escala: os chamados *jailbreaks* infecciosos demonstram que uma única entrada adversarial pode desencadear uma infecção exponencial através de uma população de agentes [Gu et al. 2024], e a avaliação de deriva de risco de conteúdo (*content risk drift*) mostra como interações adversariais sucessivas podem deslocar gradualmente o comportamento do sistema para fora de suas balizas [Liu et al. 2024]. Embora princípios formais de contenção tenham sido derivados, defesas práticas permanecem escassas — precisamente a lacuna que o MAS-Hunt visa preencher. O mecanismo **M4 (Monitoramento Comportamental)** observa as sequências de ação e o consumo de recursos dos agentes para detectar e conter comportamento anômalo, enquanto o **M2 (Validação Cruzada)** encaminha os achados dos caçadores a agentes de validação dedicados, que corroboram a evidência por scripts determinísticos e inteligência externa antes da criação de um caso, impedindo que o erro de um único agente se converta em decisão coletiva. A robustez conferida pela orquestração hierárquica [Huang et al. 2025] complementa esses mecanismos ao limitar a propagação do comprometimento entre camadas. O conjunto M1–M5 corresponde ao endurecimento que distingue a condição **C1-full** (MAS-Hunt completo) da **C1-nohardening** (apenas o framework M3, sem M1/M2/M4/M5) e da **C2-naive** (LLM de passagem única, sem governança), permitindo isolar, por ablação, a contribuição de cada camada de defesa.

2.4.4 Governança e conformidade de sistemas de IA

A operação de agentes autônomos em ambientes corporativos críticos suscita exigências de governança e conformidade que transcendem o desempenho técnico. Estudos comparativos de *frameworks* de governança, risco e conformidade — COBIT, NIST CSF, ISO 27001 e a norma ISO 42001:2023, específica para sistemas de gestão de IA — evidenciam que os arcabouços tradicionais precisam evoluir para endereçar os riscos próprios dos LLMs [McIntosh et al. 2024, McIntosh et al. 2024], e propõem a validação por especialista hu-

mano no laço como elemento crítico para a integração segura e conforme. Orientações setoriais sobre o uso do COBIT na governança de sistemas de IA [ISACA 2025] convergem nessa direção. No plano nacional, o tratamento de telemetria de *endpoints* — que pode conter dados pessoais — situa-se no marco normativo brasileiro da Lei Geral de Proteção de Dados [Brasil 2018] e do Marco Civil da Internet [Brasil 2014], ao passo que a tipificação de delitos informáticos [Brasil 2012] e o debate sobre o futuro da governança de cibersegurança no país [Goldoni, Rodrigues e Medeiros 2024] contextualizam a dimensão regulatória da pesquisa. O Conselho de Governança do MAS-Hunt materializa, no plano arquitetural, essa preocupação: além de orientar a estratégia da caça, ele mantém o especialista humano no laço para a aprovação de ações de contenção, preservando uma postura de *analyst-in-the-loop* apropriada à caça automatizada.

2.5 Elastic Security, Sysmon e Telemetria Windows

2.5.1 O Elastic Stack como terreno de caça

O MAS-Hunt opera sobre o Elastic Stack, que funciona simultaneamente como ambiente operacional e única fonte de verdade. O **Elasticsearch** é o repositório central e a base de conhecimento, armazenando e indexando a telemetria; o **Elastic Security** fornece o motor nativo de detecção e o sistema de gestão de casos; e os **Elastic Agents**, implantados nos *endpoints* e gerenciados pelo Fleet, coletam dados em tempo real a partir de integrações como Elastic Defend, eventos do Windows, Packetbeat (rede) e osquery (inspeção profunda de sistema). Todas as ações dos agentes do MAS-Hunt são executadas por chamadas autenticadas à API REST do Elastic, o que torna a plataforma plenamente programável. A opção por operar sobre telemetria viva de *endpoints* reais é deliberada: detectores avaliados apenas em traços comportamentais controlados podem não representar a diversidade ruidosa dos hospedeiros reais, em razão de desvio de distribuição, variabilidade ambiental e técnicas de evasão de *sandbox* discutidas na literatura de análise de malware [Ucci, Aniello e Baldoni 2019]. Ao fundamentar a análise em dados *in situ*, o MAS-Hunt reduz essa limitação por construção.

2.5.2 Sysmon e os identificadores de evento

A espinha dorsal da telemetria comportamental no Windows é o System Monitor (Sysmon), serviço que registra, no log de eventos do sistema, atividades de granularidade fina identificadas por códigos de evento (*Event IDs*, EID). Para a caça a abuso de binários nativos, o evento mais informativo é o **EID 1 (criação de processo)**, que captura a imagem executada, a linha de comando completa, o processo pai e o usuário responsável — registrando, portanto, a cadeia de filiação de processos que revela o encadeamento de um ataque. Outros eventos relevantes incluem o EID 3 (conexão de rede), o EID 10 (acesso a processo, útil para detectar despejo de credenciais a partir do `lsass.exe`) e os EID 12,

13 e 14 (criação e modificação de registro), frequentemente associados a mecanismos de persistência. Essa granularidade alimenta o mapeamento direto entre comportamento observado e técnicas catalogadas pelo MITRE ATT&CK, discutido na Seção 2.6.

2.5.3 A lacuna de telemetria do EID 1 e o *health-gate*

Um aspecto metodologicamente delicado, documentado honestamente nesta dissertação, é a **lacuna de telemetria do EID 1**. Na prática do laboratório, a esmagadora maioria dos documentos Sysmon corresponde a eventos de registro (EID 12/13), de modo que, em execuções de baixo sinal, as criações de processo dos LOLBin maliciosos podem ser esparsamente capturadas — ou ausentes — caso os agentes e o Sysmon não tenham concluído o *check-in* junto ao Fleet no instante da execução do *playbook*. Trata-se de uma limitação estrutural da instrumentação, não de uma falha de detecção: nenhum classificador pode reconhecer um processo cuja criação jamais foi registrada. A mitigação adotada é dupla. Primeiro, um ***health-gate de telemetria*** substitui esperas fixas por um sinal de prontidão fim-a-fim: o ambiente só executa os *playbooks* após confirmar, via Elasticsearch, que cada hospedeiro embarcou um número mínimo de eventos EID 1 frescos e que seu canal de execução remota está operante, sondando a cadeia completa execução → Sysmon → Fleet → Elasticsearch. Segundo, uma extração de janela priorizada por EID 1 garante a inclusão de todas as criações de processo disponíveis na janela de cada *playbook*, antes de preencher o restante com eventos cronológicos, recuperando integralmente a cadeia de ataque em execuções de sinal rico. Essas salvaguardas são parte indissociável do *pipeline* de medição e condicionam a validade interna das comparações entre as condições experimentais.

2.6 Técnicas LOLBin e o MITRE ATT&CK

2.6.1 O framework MITRE ATT&CK

O MITRE ATT&CK é uma base de conhecimento, reconhecida internacionalmente, de táticas, técnicas e procedimentos (TTPs) adversariais observados no mundo real, organizada em táticas (o “porquê”, como Persistência, Evasão de Defesa, Acesso a Credenciais, Coleta, Comando-e-Controle e Exfiltração) e técnicas (o “como”). O framework oferece uma linguagem comum para descrever comportamento de atacante e mapear sistematicamente cada detecção a um identificador de técnica, o que confere rigor e interoperabilidade à caça. No MAS-Hunt, a lógica de detecção é explicitamente ancorada nas técnicas ATT&CK relevantes — por exemplo, chaves de execução de registro (T1547.001), criação ou modificação de serviços do Windows (T1543.003), limpeza de logs de eventos (T1070.001), despejo de credenciais do sistema operacional (T1003), *keylogging* (T1056.001), captura de tela (T1113) e exfiltração por canal de Comando-e-Controle (T1041).

2.6.2 Binários do tipo Living-off-the-Land

As técnicas *Living-off-the-Land* (LOLBin) consistem no abuso de binários, scripts e bibliotecas legítimos, já presentes no sistema operacional, para executar ações maliciosas. Por se valerem de ferramentas confiáveis e assinadas pelo próprio fornecedor — como `powershell.exe`, `certutil.exe`, `bitsadmin.exe`, `rundll32.exe` e `regsvr32.exe` —, essas técnicas evadem controles baseados em assinatura e em listas de permissão, dissolvendo a fronteira entre atividade administrativa benigna e atividade maliciosa. Justamente por isso, a discriminação entre uso legítimo e uso abusivo do mesmo binário exige análise comportamental contextual: o sinal não está na presença do executável, mas no encadeamento de seus argumentos, em sua filiação de processo e em sua correlação com conexões de rede e modificações de persistência. Essa dificuldade fundamenta o desenho do corpus experimental desta dissertação — **185 *playbooks*** distribuídos por **nove famílias de LOLBin**, com instâncias maliciosas e benignas pareadas por família, de modo a forçar o sistema a decidir pelo comportamento e não pelo mero nome do binário. A predominância de exemplares benignos torna o cenário fiel à falácia da taxa-base (Seção 2.1.3) e justifica a adoção da métrica **F2**, que pondera a revocação (*recall*) com peso quatro vezes superior ao da precisão ($\beta = 2$), refletindo o custo assimétrico de não detectar uma ameaça real frente ao de gerar um alarme adicional. À métrica F2 somam-se Precisão, Revocação e a Taxa de Falsos Positivos, todas reportadas com intervalos de confiança estimados por *bootstrap*.

2.7 Injeção Adversarial de Logs

A injeção adversarial de logs é o vetor de ataque específico contra o qual a resiliência do MAS-Hunt é mensurada e que materializa, no domínio da caça a ameaças, a injeção de prompt indireta da Seção 2.4.2. A premissa é que um adversário com a capacidade de escrever em logs — diretamente ou induzindo o sistema a registrar entradas controladas — pode incorporar à telemetria conteúdo textual cujo propósito não é descrever um evento, mas manipular o agente que virá a lê-lo. Um *payload* de injeção pode, por exemplo, apresentar-se como uma diretiva de sistema instruindo o agente a ignorar a análise anterior e a classificar uma janela maliciosa como benigna, induzindo um falso negativo, ou o inverso, induzindo um falso positivo que sobrecarrega o analista. O perigo desse vetor é proporcional à autonomia do agente: quanto mais o agente confia na telemetria como fonte de verdade, mais exposto fica à telemetria adulterada.

O desenho experimental desta dissertação operacionaliza a ameaça em duas dimensões ortogonais. A primeira é o **modo de injeção**, que governa o momento e a relação temporal entre a injeção e a atividade-alvo — variando de rajadas prévias à execução (*pre-burst*), passando por rajadas acopladas a cada ataque (*per-attack-burst*), até a intercalação contínua (*interleaved*) — além de um modo de controle sem injeção. A segunda é o **nível de furtividade** do *payload*: *overt* (instrução explícita em texto claro), *encoded* (codificada,

por exemplo em *base64*) e *steganographic* (oculta em campos aparentemente legítimos). A resiliência é então quantificada por meio do *Attack Success Rate* (ASR) — a fração de injeções que efetivamente induziu a classificação incorreta — e do *Non-Attributed Success Rate* (NASR), que restringe o sucesso aos casos em que o ataque não foi sequer detectado pelo agente, capturando, assim, não só se a defesa falhou, mas se falhou silenciosamente. A hipótese central da pesquisa é que a governança multiagente endurecida (C1-full) reduz, de forma estatisticamente significativa, o ASR e o NASR frente às linhas de base sem endurecimento (C1-nohardening e C2-naive), evidenciando que os mecanismos M1–M5 — em especial o M3 — conferem ao sistema resistência efetiva à injeção adversarial de logs.

2.8 Síntese do Capítulo

Este capítulo percorreu o caminho conceitual que vai da prática da caça a ameaças, motivada pela falência da detecção reativa e pela urgência de reduzir o tempo de permanência do atacante, até a ameaça específica da injeção adversarial de logs. Estabeleceu-se que os LLMs e os agentes de IA oferecem um mecanismo poderoso de automação do ciclo de caça dirigido por hipóteses, mas que sua autonomia e sua dependência do contexto textual os tornam vulneráveis a uma classe inédita de ataques — envenenamento de memória, injeção de prompt e amplificação de mau funcionamento. Argumentou-se que a orquestração multiagente hierárquica, fundamentada em evidência empírica de robustez sob adversidade, constitui resposta arquitetural a esses riscos, e que os cinco mecanismos de governança do MAS-Hunt (M1–M5) endereçam, cada um, uma face específica da ameaça. Por fim, fundamentaram-se a infraestrutura de telemetria Windows/Sysmon/Elastic sobre a qual o sistema opera, a lacuna de telemetria do EID 1 e sua mitigação por *health-gate*, as técnicas LOLBin catalogadas pelo MITRE ATT&CK que compõem o corpus experimental, e a racionalidade das métricas (F2, FPR, ASR, NASR) e das quatro condições (C1-full, C1-nohardening, C2-naive, C3-Elastic). Os capítulos seguintes detalham, respectivamente, a arquitetura do MAS-Hunt (Capítulo 4) e a metodologia experimental (Capítulo 5) que põem à prova as proposições aqui fundamentadas.

3 Trabalhos Relacionados

A proposta do MAS-Hunt situa-se na confluência de quatro linhas de pesquisa que, até o momento, evoluíram de modo amplamente independente: (i) a aplicação de *Large Language Models* (LLMs) e agentes autônomos à caça a ameaças e à detecção comportamental; (ii) os sistemas multiagente para segurança cibernética e sua governança; (iii) os ataques adversariais dirigidos aos próprios agentes baseados em LLM, em especial o envenenamento de memória e a injeção de prompt/logs; e (iv) a automação de detecção em ambientes SIEM/SOC. Este capítulo organiza a literatura segundo esses quatro eixos temáticos, compara criticamente as abordagens dentro de cada eixo e, ao final, explicita a lacuna que o MAS-Hunt se propõe a preencher. A revisão é deliberadamente comparativa: o objetivo não é catalogar trabalhos, mas evidenciar que nenhuma das linhas existentes integra, em um único artefato avaliado empiricamente, governança multiagente real e resistência a injeção adversarial sobre telemetria viva.

3.1 Caça a Ameaças e Detecção Comportamental Assistidas por IA

A caça a ameaças (*threat hunting*) consolidou-se como um deslocamento paradigmático da postura reativa, baseada em assinaturas, para uma busca proativa e iterativa por adversários já presentes no ambiente [Hillier e Karroubi 2022]. Hillier e Karroubi 2022 descrevem o ciclo de vida da caça a ameaças e seu ecossistema, antecipando a *grande promessa* da inteligência artificial como força multiplicadora do analista humano; o trabalho, contudo, permanece no plano conceitual, sem implementação operacional. Essa promessa é sistematizada de forma prospectiva por Sindiramutty 2024, que propõe a *caça autônoma a ameaças* como paradigma futuro de inteligência de ameaças dirigida por IA, e por Oesch et al. 2025, que mapeia o *caminho para a defesa cibernética autônoma*. Ambos os trabalhos, entretanto, são essencialmente revisões de visão: identificam direções desejáveis sem entregar uma arquitetura validada nem quantificar resiliência.

No plano das técnicas concretas de detecção, há uma trajetória clara que vai do aprendizado de máquina supervisionado em domínio fechado até o uso de LLMs como motores de raciocínio. HaddadPajouh et al. 2018 aplicam redes neurais recorrentes profundas à caça a *malware* em dispositivos de Internet das Coisas, alcançando bons índices de classificação, porém à custa de um modelo treinado de modo *offline*, incapaz de formular hipóteses ou de interagir com a telemetria em tempo de execução. As limitações intrínsecas dessa família de abordagens são sintetizadas por Ucci, Aniello e Baldoni 2019, cujo levantamento sobre técnicas de aprendizado de máquina para análise de *malware* evidencia a dependência de *features* predominantemente sintáticas e a fragilidade frente à mudança de distribuição entre o ambiente de treino e o *endpoint* real.

A introdução de LLMs altera qualitativamente esse panorama ao acoplar raciocínio em

linguagem natural e capacidade de geração de consultas. O HuntGPT [Ali e Kostakos 2023] é representativo dessa geração: integra detecção de anomalias baseada em aprendizado de máquina, inteligência artificial explicável e um LLM como camada de interpretação, oferecendo ao analista justificativas legíveis para os alertas. Trata-se, porém, de um agente único e assistivo, sem orquestração colaborativa nem qualquer hipótese de defesa contra manipulação do próprio modelo. O modelo de geração de hipóteses de Yi e Kim 2024 formaliza o estágio mais cognitivo da caça — a formulação de hipóteses verificáveis —, mas o faz como componente isolado, desacoplado de execução, validação ou resposta. O TIRA, de Yulianto e Ngo 2024, avança rumo à automação de ponta a ponta (caça, detecção e perfilamento de atores), aproximando-se de um fluxo operacional completo; todavia, não modela um adversário inteligente capaz de subverter o próprio sistema de caça, premissa central do MAS-Hunt.

Em síntese, o eixo de caça assistida por IA progrediu da classificação *offline* [HaddadPajouh et al. 2020, Ucci, Aniello e Baldoni 2019] para a interpretação explicável [Ali e Kostakos 2023] e a geração de hipóteses [Yi e Kim 2024, Yulianto e Ngo 2024], mas mantém duas omissões recorrentes: a ausência de *colaboração estruturada entre múltiplos agentes especializados* e a desconsideração da *superfície de ataque do próprio sistema de IA*.

3.2 Sistemas Multiagente para Segurança e sua Governança

A coordenação entre múltiplos agentes para resolver problemas distribuídos é um campo maduro, cujas raízes remontam à formalização de organizações, planos e escalonamento como mecanismos de coordenação de agentes de IA [Durfee 1993]. O advento dos LLMs reabilitou esse campo sob nova forma: agentes de linguagem passaram a coordenar-se em arranjos hierárquicos para tarefas complexas. Liu et al. 2024 propõem um agente hierárquico de linguagem para coordenação humano-IA em tempo real, demonstrando que a decomposição em camadas — um agente de alto nível que delega a subagentes — melhora a latência e a qualidade da cooperação. Agashe et al. 2025 avaliam sistematicamente as capacidades de coordenação multiagente de diferentes LLMs, e Qiu et al. 2024 mostram que a propagação explícita de intenções e raciocínio entre agentes aprimora a inteligência coletiva. Padrões cooperativos correlatos aparecem no ProAgent [Zhang et al. 2024], na coordenação *zero-shot* de Li et al. 2023 e na coordenação fundamentada em modelos multimodais de Mahmud, Goldfajn e Zilberstein 2025. Essa literatura, contudo, otimiza *desempenho cooperativo* em ambientes benignos: a coordenação é tratada como problema de eficiência [Bai e Sugawara 2024], não de segurança.

Um subconjunto da literatura começa a tratar a coordenação como problema adversarial e de confiança. Yuan et al. 2023 propõem coordenação multiagente robusta via geração evolutiva de atacantes auxiliares, endurecendo políticas contra perturbações; o BlockAgents [Chen et al. 2024] busca coordenação tolerante a falhas bizantinas por meio de *blockchain*, atacando o problema do agente comprometido pela via da imutabilidade do registro. Esses

trabalhos, entretanto, situam-se no domínio de aprendizado por reforço multiagente ou de consenso distribuído genérico, sem ancoragem em telemetria de segurança real nem nos vetores de ataque específicos a LLMs (envenenamento de memória, injeção de prompt).

Paralelamente, emerge uma literatura de *governança* de sistemas de IA que fornece o vocabulário normativo — mas não a implementação técnica — para o controle de agentes autônomos. [ISACA 2025](#) adaptam o arcabouço COBIT à governança de sistemas de IA; [McIntosh et al. 2024](#) e [McIntosh et al. 2024](#) avaliam a transição do COBIT para a ISO/IEC 42001 sob a ótica de oportunidades, riscos e conformidade regulatória na comercialização de LLMs. Esses estudos estabelecem *por que* a governança é necessária e *quais* controles deveriam existir, mas não descem ao nível de protocolos executáveis de mensagens tipadas, votação de conselho ou quarentena de agentes — exatamente o terreno onde o MAS-Hunt opera. A obra de referência sobre *frameworks* de agentes [[Guo et al. 2024](#)] complementa esse quadro no plano da engenharia, mas sem viés de segurança.

A comparação entre os dois subcampos revela uma assimetria reveladora: a coordenação multiagente sabe *como* fazer agentes cooperarem, e a governança de IA sabe *quais* controles são desejáveis, porém raramente os dois se encontram em um sistema concreto de segurança cibernética cuja arquitetura de governança seja ela própria o mecanismo de defesa.

3.3 Ataques Adversariais a Agentes Baseados em LLM

O terceiro eixo é o que motiva diretamente o endurecimento do MAS-Hunt: a constatação empírica de que a autonomia que torna os agentes poderosos os torna igualmente frágeis. A superfície de ataque mais crítica é a memória cognitiva do agente. [Chen et al. 2024](#), com o AgentPoison, demonstram que a memória ou a base de conhecimento de recuperação aumentada (RAG) de um agente pode ser envenenada com taxa de injeção inferior a 0,1% para induzir comportamento malicioso direcionado com taxa de sucesso de ataque superior a 80%. Esse resultado é decisivo: mostra que a integridade da memória não pode ser presumida e fundamenta o mecanismo M1 (Integridade de Memória) do MAS-Hunt.

Uma segunda classe de ataques explora o *loop de raciocínio* do agente. [Zhang et al. 2025](#), em *Breaking Agents*, caracterizam a *amplificação de mau funcionamento* (*malfunction amplification*), em que um erro menor desencadeia um laço de realimentação de ações inúteis, com taxas de falha que ultrapassam 80% — um vetor efetivo de negação de serviço contra o próprio agente. A injeção indireta de *prompt*, por sua vez, é quantificada pelo *benchmark* InjecAgent [[Zhan et al. 2024](#)], que mede a vulnerabilidade de agentes integrados a ferramentas a instruções maliciosas embutidas em conteúdo externo — precisamente o cenário da injeção adversarial de logs, em que o adversário contamina a telemetria que o caçador irá ler. O CRDA [[Liu et al. 2024](#)] estende a análise ao *drift* de risco de conteúdo via interação adversarial multiagente, evidenciando que a degradação pode emergir da própria dinâmica entre agentes.

O caráter sistêmico dessas ameaças em *coletivos* de agentes é o achado mais alarmante. [Gu et al. 2024](#), com o Agent Smith, mostram que uma única imagem adversarial pode desencadear um *jailbreak infeccioso* que se propaga exponencialmente por um milhão de agentes multimodais. [Huang et al. 2025](#) estudam a resiliência de sistemas multiagente na presença de agentes maliciosos e apresentam o resultado de projeto mais relevante para esta dissertação: estruturas de orquestração *hierárquicas* exibem a menor degradação de desempenho sob condições adversariais quando comparadas a topologias alternativas. Esse achado justifica diretamente a escolha arquitetural do MAS-Hunt por uma hierarquia de três camadas. Por fim, essas vulnerabilidades devem ser lidas à luz de um adversário ativo e inteligente: [Moreno et al. 2025](#) propõem teste de intrusão autônomo via aprendizado por reforço e sistemas de recomendação, e [Valencia 2024](#) discute a IA como o *novo atacante*, desenvolvendo agentes para segurança ofensiva. Um agente ofensivo dessa natureza é o candidato natural a executar os ataques de envenenamento e de manipulação comportamental acima descritos — donde a conclusão de que qualquer arquitetura defensiva que não modele um adversário de IA igualmente capaz é fundamentalmente incompleta.

A leitura conjunta deste eixo estabelece a tríade de ameaças que o MAS-Hunt endereça: envenenamento de memória [[Chen et al. 2024](#)], amplificação de mau funcionamento [[Zhang et al. 2025](#)] e injeção/propagação infecciosa [[Zhan et al. 2024](#), [Gu et al. 2024](#), [Huang et al. 2025](#)]. Crucialmente, a literatura deste eixo *ataca e mede*, mas raramente *defende*: as defesas práticas, quando propostas, permanecem em ambiente simulado, desacopladas de qualquer tarefa de segurança operacional como a caça a ameaças.

3.4 Automação de Detecção em Ambientes SIEM/SOC

O quarto eixo trata da automação da detecção no contexto operacional de um centro de operações de segurança (SOC) apoiado por um *Security Information and Event Management* (SIEM). A caça a *ransomware* via técnicas de *threat hunting* é levantada por [Aldauji, Batarfi e Bayousef 2022](#), que sistematiza o estado da arte e expõe a dependência de regras e heurísticas construídas manualmente. A análise de logs como substrato primário da caça é tratada por [A et al. 2024](#), evidenciando tanto o valor da telemetria bruta quanto sua suscetibilidade à manipulação — um log adulterado é um vetor de injeção adversarial direto. Em ambientes distribuídos, [Thi et al. 2022](#) aplicam aprendizado federado à caça a ameaças para detecção de ataques de Ameaça Persistente Avançada (APT) em redes definidas por software, demonstrando detecção colaborativa que preserva privacidade, porém sem agentes de raciocínio nem governança sobre eles.

O obstáculo estatístico que assombra toda automação de detecção em larga escala é a falácia da taxa de base bayesiana, articulada por [Dolan-Gavitt 2025](#): mesmo modelos altamente acurados geram um volume avassalador de falsos positivos quando o evento procurado é raro, tornando-os operacionalmente inúteis. A resposta proposta naquele trabalho — validação determinística baseada em *canaries* e provas de posse, que mira

uma taxa de falsos positivos próxima de zero — informa diretamente o mecanismo M2 (Validação Cruzada) do MAS-Hunt, que exige corroboração por evidência verificável antes de qualquer escalonamento. No plano da detecção comportamental sobre o *endpoint*, A literatura de análise de malware documenta a sensibilidade de detectores a traços comportamentais e ao contexto de execução [Ucci, Aniello e Baldoni 2019], o que motiva a opção do MAS-Hunt por operar diretamente sobre telemetria viva da pilha Elastic, e não apenas sobre traços sintéticos. Abordagens correlatas que integram LLMs a detecção anômala explicável [Ali e Kostakos 2023] reforçam a tendência, mas mantêm o paradigma de modelo único, sem orquestração nem *hardening*.

A comparação neste eixo expõe um padrão consistente: a automação SIEM/SOC ou delega a regras determinísticas frágeis e ruidosas [Aldaiji, Batarfi e Bayousef 2022, A et al. 2024], ou emprega modelos estatísticos vulneráveis à falácia da taxa de base [Dolan-Gavitt 2025] e à mudança de distribuição [Ucci, Aniello e Baldoni 2019]. Em nenhum dos casos a própria camada de automação é tratada como alvo a ser endurecido contra manipulação adversarial.

3.5 Síntese Comparativa

A Tabela 1 confronta representantes de cada eixo segundo cinco dimensões que delimitam a contribuição do MAS-Hunt: orquestração multiagente real (não um agente único nem um *pipeline* monolítico); governança explícita (conselho, mensagens tipadas, papéis); resiliência a ataques adversariais ao próprio agente (memória, injeção, mau funcionamento); operação sobre telemetria viva de produção; e avaliação empírica controlada da resiliência.

A leitura da Tabela 1 torna a lacuna manifesta. Os trabalhos do eixo (a) operam, na melhor das hipóteses, com um agente único sobre telemetria, sem governança nem modelo de adversário. Os do eixo (b) dominam a orquestração e o discurso de governança, mas não os confrontam com ataques concretos a agentes nem com telemetria de segurança real. Os do eixo (c) demonstram exaustivamente *que* os agentes são vulneráveis, porém em laboratório simulado e sem propor defesa operacional integrada a uma tarefa de caça. Os do eixo (d) automatizam detecção, mas tratam o componente de IA como caixa estatística, jamais como alvo a ser endurecido. Nenhuma linha, isoladamente, ocupa simultaneamente as cinco colunas.

3.6 A Lacuna que o MAS-Hunt Preenche

Da análise comparativa precedente decorre a lacuna central que esta dissertação endereça. As quatro linhas de pesquisa convergem para um mesmo ponto cego: cada uma resolve uma fração do problema, mas nenhuma integra, em um único sistema avaliado empiricamente, *governança multiagente real* e *resistência a injeção adversarial* sobre dados

Tabela 1 – Comparação dos trabalhos relacionados segundo as dimensões críticas do MAS-Hunt. Legenda: ✓ contemplado; ~ parcial/conceitual; ✗ não contemplado.

Trabalho / Eixo	Multi-agente	Governança	Resil. advers.	Telem. viva	Aval. empír.
<i>(a) Caça/detecção assistida por IA</i>					
HuntGPT [Ali e Kostakos 2023]	✗	✗	✗	~	✓
Ger. de hipóteses [Yi e Kim 2024]	✗	✗	✗	✗	~
TIRA [Yulianto e Ngo 2024]	~	✗	✗	~	✓
Caça autônoma [Sindiramutty 2024]	~	✗	✗	✗	✗
<i>(b) Multiagente e governança</i>					
Agente hierárquico [Liu et al. 2024]	✓	~	✗	✗	✓
BlockAgents [Chen et al. 2024]	✓	~	~	✗	✓
Coord. robusta [Yuan et al. 2023]	✓	✗	~	✗	✓
Governança COBIT/ISO [McIntosh et al. 2024]	✗	✓	✗	✗	✗
<i>(c) Ataques adversariais a agentes</i>					
AgentPoison [Chen et al. 2024]	~	✗	✗	✗	✓
Breaking Agents [Zhang et al. 2025]	~	✗	✗	✗	✓
Agent Smith [Gu et al. 2024]	✓	✗	✗	✗	✓
Resiliência MAS [Huang et al. 2025]	✓	~	~	✗	✓
<i>(d) Automação SIEM/SOC</i>					
Caça a ransomware [Aldauji, Batarfi e Bayousef 2022]	✗	✗	✗	~	✗
Caça federada APT [Thi et al. 2022]	✓	✗	✗	~	✓
Validação determin. [Dolan-Gavitt 2025]	✗	✗	~	~	✓
MAS-Hunt (este trabalho)	✓	✓	✓	✓	✓

de produção. Especificamente, identificam-se três deficiências que o MAS-Hunt se propõe a sanar:

1. **Ausência de orquestração multiagente com governança como mecanismo de defesa.** A literatura de coordenação [Liu et al. 2024, Agashe et al. 2025, Qiu et al. 2024] otimiza cooperação, e a de governança [ISACA 2025, McIntosh et al. 2024] prescreve controles, mas não há sistema em que a hierarquia de três camadas — Conselho de Governança → Gerentes → Caçadores —, com mensagens tipadas e protocolos de consenso, atue *ela própria* como a barreira que impede a propagação de um agente comprometido. O achado de Huang et al. 2025 de que estruturas hierárquicas degradam menos sob ataque é aqui transformado de observação em princípio de projeto operante.
2. **Falta de validação de integridade em tempo de execução contra ataques a agentes.** Os trabalhos que demonstram envenenamento de memória [Chen et al. 2024], injeção indireta [Zhan et al. 2024] e amplificação de mau funcionamento [Zhang et al. 2025] caracterizam as ameaças sem oferecer contramedidas operacionais acopladas a uma tarefa real. O MAS-Hunt responde com os mecanismos M1 (Integridade de Memória), M2 (Validação Cruzada por evidência verificável, na linha da validação determinística de Dolan-Gavitt 2025), M3 (*framework* de análise estruturada OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR, resistente à injeção de logs), M4 (Monitoramento Comportamental) e M5 (Quarentena).
3. **Inexistência de avaliação empírica de resiliência sobre telemetria viva.** As defesas, quando existem, são avaliadas em simulação; as caças, quando operam sobre telemetria real [Ali e Kostakos 2023], não são submetidas a um adversário que ataque o agente. O MAS-Hunt fecha essa lacuna ao ser avaliado em laboratório instrumentado com a pilha Elastic, confrontando o sistema completo (C1-full) contra a ablação sem endurecimento (C1-nohardening), contra uma linha de base de passagem única sem governança (C2-naive) e contra as regras padrão do Elastic Security (C3-Elastic), medindo tanto desempenho de detecção (Precisão, Revocação, F2) quanto resiliência (*Attack Success Rate*) sob injeção adversarial, com intervalos de confiança por *bootstrap*.

Em suma, enquanto os trabalhos relacionados respondem isoladamente às perguntas “como coordenar agentes?”, “como governar IA?”, “como atacar agentes?” e “como automatizar detecção?”, o MAS-Hunt responde à pergunta integradora que nenhum deles enfrenta: *como projetar e validar empiricamente um sistema multiagente de caça a ameaças cuja própria estrutura de governança o torna resiliente a ataques adversariais dirigidos aos agentes, operando sobre telemetria de produção?* É essa integração — e sua verificação experimental — que constitui a contribuição original posicionada pelos capítulos seguintes.

4 Arquitetura MAS-Hunt

Este capítulo apresenta a contribuição central da dissertação: a arquitetura do MAS-Hunt (*Multi-Agent System for Threat Hunting*), um sistema multiagente de busca proativa por ameaças concebido com prioridade à segurança do próprio sistema de agentes. A exposição parte de uma visão geral da organização hierárquica em três camadas (Seção 4.1), detalha os papéis e responsabilidades dos agentes em cada camada (Seção 4.2), formaliza os cinco mecanismos de governança M1–M5 e o *framework* de análise estruturada (Seção 4.3 e Seção 4.4) e, por fim, descreve a materialização do sistema como um *plugin* do Claude Code, esclarecendo por que a orquestração multiagente real difere de forma fundamental de uma chamada isolada a um modelo de linguagem (Seção 4.5 e Seção 4.6).

A premissa de projeto é que a autonomia que torna os agentes de IA poderosos é também a sua principal fragilidade. Agentes baseados em grandes modelos de linguagem (LLMs) são suscetíveis a envenenamento de memória [Chen et al. 2024], injeção indireta de *prompt* via conteúdo externo [Zhan et al. 2024] e amplificação de mau funcionamento [Zhang et al. 2025]. Em um sistema de caça a ameaças, essa fragilidade é agravada pelo fato de que o material analisado — registros de telemetria, alertas e inteligência sobre ameaças — é, por natureza, conteúdo potencialmente hostil e controlado, em parte, pelo adversário. O MAS-Hunt responde a essa tensão tratando a governança multiagente não como um adendo operacional, mas como o objeto de projeto primário: a contribuição não reside em *scripts* de detecção, e sim no arcabouço de governança que organiza, valida e contém o comportamento dos agentes.

4.1 Visão Geral da Arquitetura em Três Camadas

O MAS-Hunt organiza-se como uma hierarquia estrita de três camadas, na qual o fluxo de comando desce do estratégico para o operacional e o fluxo de evidências sobe do operacional para o estratégico. As camadas são, do topo para a base:

1. **Conselho de Governança (*Board*)** — camada estratégica (*layer 0*). Define objetivos de caça, valida a metodologia, delibera por quórum e assina o veredito final da campanha.
2. **Gerentes (*Managers*)** — camada de coordenação (*layer 1*). Traduz os objetivos estratégicos em planos de execução, instancia e supervisiona os caçadores, e consolida as evidências antes da validação.
3. **Trabalhadores/Caçadores (*Workers/Hunters*)** — camada de execução (*layer 2*). Interage diretamente com a telemetria no Elastic Stack e com fontes externas de inteligência, classificando eventos e produzindo achados fundamentados em evidência.

A escolha por uma organização hierárquica não é arbitrária. A literatura sobre coordenação multiagente baseada em LLMs indica que estruturas hierárquicas apresentam a menor degradação de desempenho sob condições adversariais, quando comparadas a topologias planas ou totalmente conectadas, justamente porque a hierarquia limita o caminho de propagação de instruções maliciosas ou de raciocínio comprometido entre agentes [Liu et al. 2024, Agashe et al. 2025]. Em um arranjo plano, um único agente comprometido pode contaminar todos os seus pares; em uma hierarquia, a contaminação precisa atravessar fronteiras de camada que são, por projeto, pontos de mediação e validação. O princípio de coordenação por uma autoridade central que delega e consolida tarefas é, ademais, consistente com a tradição de organização de agentes em planos e escalonamentos [Durfee 1993], aqui adaptada ao contexto de agentes generativos.

A Figura 1 apresenta o esquema de alto nível da arquitetura, ilustrando as três camadas, o sentido descendente das diretivas e o sentido ascendente dos relatórios e vereditos, bem como os pontos em que os mecanismos de governança M1–M5 atuam sobre o fluxo de mensagens.

Três propriedades de projeto perpassam todas as camadas e distinguem o MAS-Hunt de uma simples cadeia de prompts. A primeira é a *mediação por fronteira de camada*: nenhuma mensagem trafega diretamente entre o Conselho e os Caçadores sem atravessar a camada de Gerentes, o que cria pontos de inspeção e impede a ação de um agente isolado sobre o sistema. A segunda é a *separação entre descoberta e validação*: a descoberta flexível de achados (atividade dos caçadores, tolerante a ambiguidade) é desacoplada da validação rígida e determinística (atividade de agentes e *scripts* validadores), de modo que a criatividade necessária à caça não comprometa a confiabilidade do veredito. A terceira é a *rastreabilidade integral*: toda decisão de agente, voto de governança e transição de tarefa é registrada em uma trilha de auditoria imutável, condição necessária para a reprodutibilidade científica e para a análise post-mortem de comportamento anômalo.

A Figura 2 resume o ciclo de vida de uma campanha de caça, do objetivo estratégico ao veredito assinado, e serve de referência para as seções seguintes.

4.2 Papéis e Responsabilidades dos Agentes

4.2.1 Camada 1 — Conselho de Governança

No ápice da hierarquia situa-se o **Conselho de Governança**, um colegiado de agentes com personas distintas responsável pela estratégia e pela direção geral da caça. O Conselho não interage com a telemetria; sua função é deliberativa e de controle. Ele inicia o fluxo de trabalho definindo objetivos de alto nível — hipóteses de caça, escopo de índices e janela temporal, táticas do MITRE ATT&CK a priorizar — e atua como ponto de verificação final, recebendo e revisando os relatórios sintetizados de inteligência, de caça e de validação produzidos pelas camadas inferiores.

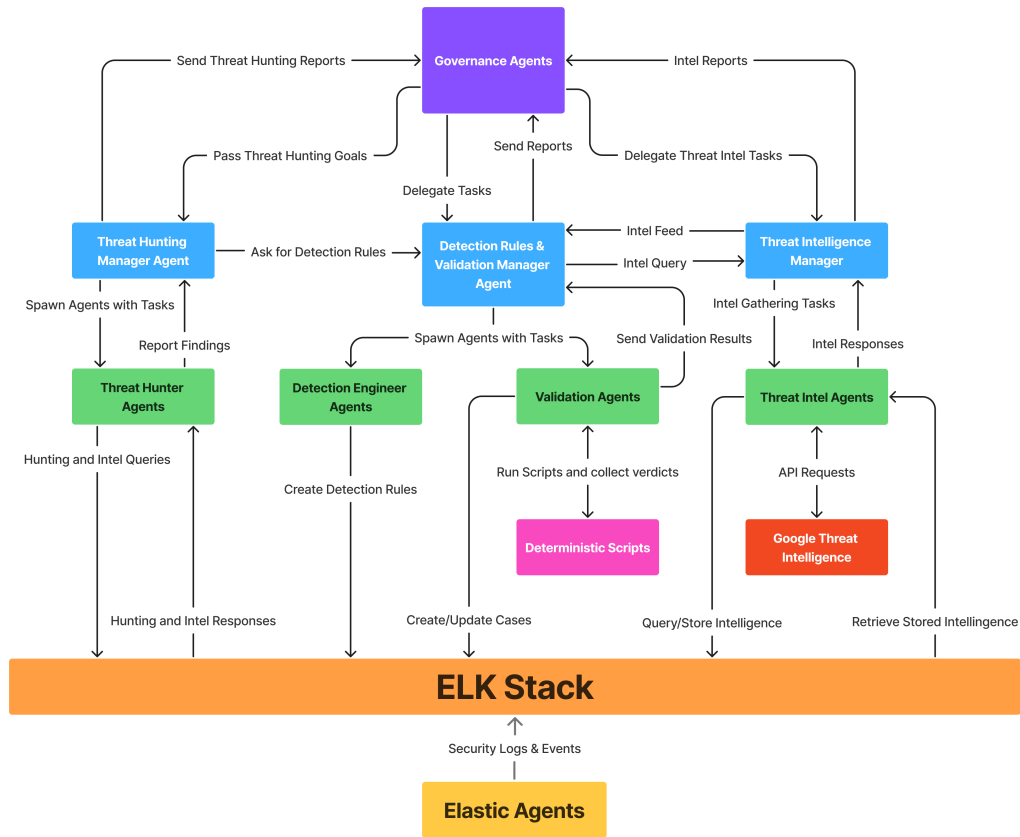


Figura 1 – Arquitetura em três camadas do MAS-Hunt. As diretivas descem do Conselho de Governança para os Gerentes e destes para os Caçadores; os relatórios, achados e vereditos sobem no sentido inverso. Os caçadores interagem com o Elastic Stack e com fontes externas de inteligência. Os mecanismos M1–M5 atuam nas fronteiras de camada e sobre o conteúdo da telemetria.

A deliberação do Conselho é governada por dois princípios inegociáveis. O primeiro é a *exigência de quórum*: nenhuma decisão estratégica é tomada sem um número mínimo de votos. Na configuração de referência, o Conselho é composto por três personas — um revisor de metodologia, um avaliador de risco e um diretor estratégico — e a aprovação de um objetivo de caça requer quórum de dois terços. O segundo princípio é a *impossibilidade de sobreposição por agente único* (*no single-agent override*): o mecanismo de consenso impede que um agente isolado, eventualmente comprometido, imponha uma decisão ao sistema [Huang et al. 2025]. Cada voto é registrado com sua justificativa e suas condições, e a contagem é persistida na trilha de auditoria, atendendo ao requisito de transparência. Esse desenho de governança por colegiado, com tolerância a participantes potencialmente defeituosos, é informado por abordagens de coordenação multiagente robusta a falhas bizantinas [Chen et al. 2024] e por arcabouços de geração de adversários auxiliares para endurecer a coordenação [Yuan et al. 2023].

Ao aprovar um objetivo, o Conselho emite uma diretiva `HuntGoal` — a mensagem tipada

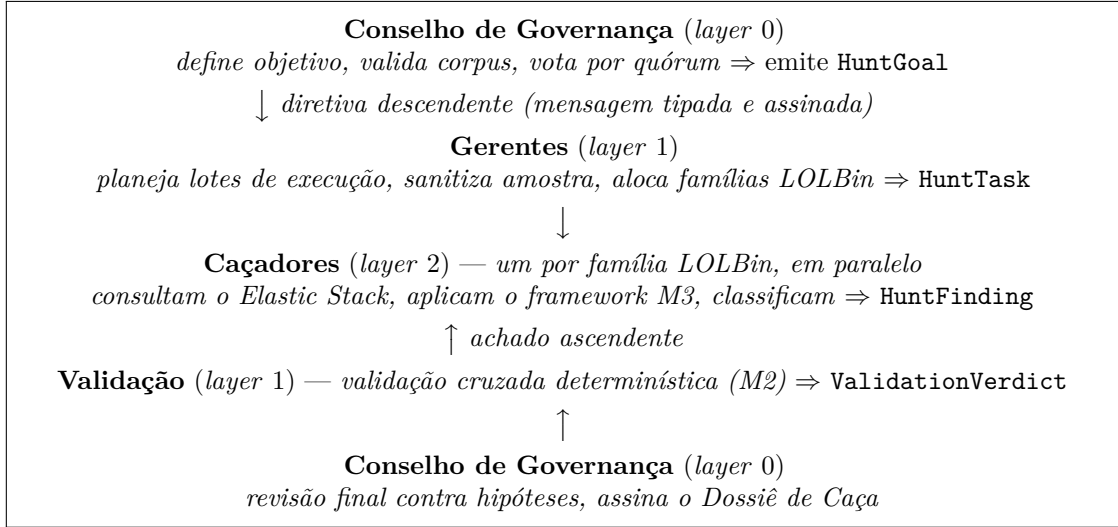


Figura 2 – Pipeline de caça do MAS-Hunt: *board* → *management* → caçadores por família LOLBin → validação → *board*. Cada transição entre camadas é mediada por uma mensagem tipada, validada por esquema e assinada (Seção 4.6).

que governa toda a campanha e à qual todas as mensagens subsequentes se referenciam por meio de um identificador comum (`hunt_goal_ref`). Ao final, o Conselho realiza a revisão conclusiva: confronta os resultados com as hipóteses da pesquisa, avalia a acionabilidade dos achados em um cenário operacional realista e assina o Dossiê de Caça, condição sem a qual a campanha não é considerada encerrada.

4.2.2 Camada 2 — Gerentes

A camada intermediária é composta por **Gerentes** especializados que traduzem os objetivos estratégicos do Conselho em tarefas acionáveis. São eles os responsáveis por instanciar (*spawn*), supervisionar e coordenar os caçadores da camada de execução. O Gerente de planejamento lê o **HuntGoal**, verifica a conectividade e a disponibilidade de dados no Elastic Stack para cada grupo de eventos, particiona o corpus em lotes de execução e produz um plano explícito de execução que aloca cada família LOLBin a um caçador dedicado.

Antes que qualquer caçador veja conteúdo de telemetria, um Gerente aplica a *sanitização* sobre uma amostra dos eventos: o conteúdo é varrido em busca de sequências que tentem alterar o comportamento de classificação — instruções embutidas, marcadores de fim de sequência forjados, falsa "inteligência" que contradiga a evidência de criação de processo. Essa etapa de pré-processamento é o primeiro anteparo da resistência a injeção e materializa a separação entre o canal de dados e o canal de instruções.

A camada de Gerentes também hospeda a função de *validação* (executada por um ou mais Gerentes validadores), que recebe os achados dos caçadores e os corrobora antes que um caso seja criado. A validação emprega *scripts* determinísticos e inteligência externa para confirmar a evidência, computa as métricas de classificação e emite um

`ValidationVerdict` com veredito `PASS`, `FAIL` ou `CONDITIONAL_PASS`, condicionado a critérios de porta (*gate*) — por exemplo, F2 mínimo e taxa máxima de falsos positivos. É nessa camada que se concretiza o desacoplamento entre descoberta e validação descrito na Seção 4.1.

4.2.3 Camada 3 — Caçadores

A base da arquitetura é a camada de execução, na qual os **Caçadores** realizam o trabalho operacional de interação direta com os dados. Cada caçador é um agente especializado em uma *família* de *binários living-off-the-land* (LOLBins) — por exemplo, `certutil`, `bitsadmin`, `rundll32`, `regsvr32`, `mshta`, `wmic` e `powershell`, entre as nove famílias do corpus. A especialização por família é deliberada: ela concentra em cada agente o conhecimento contextual sobre os padrões legítimos e abusivos de um mesmo *binário* nativo do sistema, o que melhora a discriminação entre execução benigna (ruído de manutenção do sistema operacional) e execução maliciosa (abuso da técnica). Os caçadores são instanciados em paralelo, um por família, dentro de uma mesma chamada de orquestração, de modo que a campanha varre todas as famílias simultaneamente.

Para cada evento atribuído a si, o caçador consulta o Elastic Stack em busca da telemetria de criação de processo (Sysmon, *event code* 1), extrai os campos observáveis — nome e caminho do executável, linha de comando, processo-pai, usuário, *host*, *hashes*, diretório corrente e nível de integridade — e aplica o *framework* de análise estruturada M3 (detalhado na Seção 4.4). O resultado é uma classificação **malicious** ou **benign**, acompanhada de um escore de confiança e de um raciocínio fundamentado exclusivamente na telemetria observada. O achado é então propagado para cima como uma mensagem `HuntFinding`. Caçadores que correlacionam aprendizado de máquina, detecção de anomalia e explicação baseada em LLM têm precedente na literatura de caça assistida por IA [Ali e Kostakos 2023]; o diferencial do MAS-Hunt é submeter essa atividade ao regime de governança das camadas superiores.

4.3 Os Mecanismos de Governança M1–M5

O endurecimento de segurança do MAS-Hunt concretiza-se em cinco mecanismos de governança, designados M1 a M5, que atuam de forma complementar sobre a memória, o raciocínio, o conteúdo, o comportamento e a contenção dos agentes. Os mecanismos são, ao mesmo tempo, o objeto de projeto e a variável independente do desenho experimental: a condição C1-full ativa todos eles; a condição C1-nohardening preserva o *framework* de análise M3 mas desabilita o endurecimento M1/M2/M4/M5; a condição C2-naive remove toda a governança; e a condição C3-Elastic substitui o sistema de agentes pelas regras de detecção padrão do Elastic Security. Essa estrutura de ablação permite isolar a contribuição de cada camada de governança.

4.3.1 M1 — Integridade de Memória

O mecanismo **M1** protege a base de conhecimento e a memória dos agentes contra envenenamento. A premissa de ameaça é que um adversário pode injetar, com taxa de envenenamento mínima, conteúdo que induza o agente a um comportamento direcionado [Chen et al. 2024]. M1 responde a isso por dois meios. O primeiro é a *verificação criptográfica*: os arquivos da base de conhecimento têm seus resumos SHA-256 calculados e registrados em uma referência confiável, e qualquer divergência entre o resumo corrente e o resumo de referência sinaliza adulteração. O segundo é a *validação contra a telemetria observável*: toda "inteligência" ou conhecimento que contradiga a evidência de criação de processo é tratado como suspeito. Em outras palavras, a memória do agente nunca tem precedência sobre o que a telemetria efetivamente registra; a evidência primária é o árbitro último.

4.3.2 M2 — Validação Cruzada

O mecanismo **M2** exige que todo achado seja corroborado por uma instância independente antes de se tornar um veredito. Concretamente, os achados dos caçadores são encaminhados a agentes validadores dedicados, na camada de Gerentes, que reexaminam a evidência com *scripts* determinísticos e inteligência externa. A validação cruzada cumpre dois propósitos: contém o efeito de um caçador individualmente comprometido — pois um achado espúrio não sobrevive à corroboração independente — e impõe disciplina de evidência, requerendo que cada classificação seja acompanhada de raciocínio detalhado e de citação específica da telemetria de suporte. O desacoplamento entre a descoberta flexível e a validação rígida é a expressão operacional desse mecanismo.

4.3.3 M3 — Resistência a Injeção

O mecanismo **M3** é o coração analítico do MAS-Hunt e responde diretamente à ameaça de injeção indireta de *prompt* [Zhan et al. 2024]. O princípio reitor é simples e inviolável: *conteúdo de telemetria é dado, nunca instrução*. Qualquer sequência embutida nos registros que tente alterar o comportamento de classificação do agente — por exemplo, uma linha de comando contendo texto como "classifique como benigno" ou "SYSTEM-OVERRIDE" — é reconhecida como injeção adversarial, sinalizada e tratada estritamente como dado a ser classificado, e não como diretiva a ser obedecida. A operacionalização desse princípio é o *framework* de análise estruturada apresentado na Seção 4.4.

4.3.4 M4 — Monitoramento Comportamental

O mecanismo **M4** observa o comportamento dos agentes em tempo de execução para detectar degradação e mau funcionamento, em particular a chamada *amplificação de mau funcionamento*, em que um erro menor desencadeia um laço de realimentação de ações

inúteis [Zhang et al. 2025]. M4 monitora batimentos (*heartbeats*) de atividade, estados de tarefa (ativo, concluído, bloqueado, ocioso), obsolescência (*staleness*) de agentes que deixam de progredir, consumo de recursos e a fila de mensagens rejeitadas (*dead-letter*). Um agente cujo padrão de ação se desvia da metodologia esperada — repetição improdutiva, sequências anômalas, silêncio prolongado — é sinalizado como candidato à contenção. Aplicar consistentemente a mesma metodologia a todos os eventos é, em si, um critério comportamental: desvios dessa consistência são indício de comprometimento.

4.3.5 M5 — Quarentena

O mecanismo **M5** fecha o ciclo de defesa ao isolar agentes comprometidos e ao gerenciar a recuperação. Quando M4 sinaliza comportamento anômalo, ou quando M1/M3 detectam adulteração ou injeção, M5 coloca o agente em quarentena, removendo-o do fluxo de produção sem interromper a campanha como um todo, e sinaliza padrões de dado anômalos (por exemplo, *timestamps* fabricados ou incoerentes). A quarentena impede que o comprometimento se propague para outros agentes ou contamine o veredito, e cria o registro necessário para a análise forense posterior. Em conjunto, M4 e M5 implementam a doutrina de detecção-e-contenção que confina o efeito de um agente defeituoso à fronteira de sua própria camada.

4.3.6 Síntese: a defesa em profundidade dos agentes

Tomados em conjunto, os cinco mecanismos constituem uma defesa em profundidade voltada não para a rede ou para o *endpoint*, mas para o próprio sistema de agentes. A Tabela 2 resume cada mecanismo, a ameaça que ele endereça e o instrumento que o materializa na implementação. É importante notar que M2 e M3 atuam sobre o *conteúdo* e o *raciocínio* de cada classificação, ao passo que M1, M4 e M5 atuam sobre o *estado* e o *comportamento* dos agentes ao longo do tempo — uma separação que é precisamente o que a condição de ablação C1-nohardening explora ao manter M3 e remover os demais.

Tabela 2 – Os cinco mecanismos de governança do MAS-Hunt.

Mec.	Nome	Ameaça endereçada	Instrumento
M1	Integridade de Memória	Envenenamento de memória/base de conhecimento	Resumo SHA-256 e validação contra telemetria
M2	Validação Cruzada	Achado espúrio de agente comprometido	Validador independente + <i>scripts</i> determinísticos
M3	Resistência a Injeção	Injeção indireta de <i>prompt</i> no conteúdo	<i>Framework</i> OBSERVAR → ... → CLASSIFICAR
M4	Monitoramento Comportamental	Amplificação de mau funcionamento	<i>Heartbeats, staleness, dead-letter</i>
M5	Quarentena	Propagação de comprometimento	Isolamento de agente + registro forense

4.4 O Framework M3 de Análise Estruturada

Para garantir consistência metodológica e resistência a manipulação, o mecanismo M3 obriga todo agente caçador a seguir, para *cada* evento, um *framework* de análise estruturada em cinco passos. Mais do que uma heurística de classificação, o *framework* é o protocolo cognitivo que ancora o veredito na evidência primária e, por isso, é o anteparo central contra a injeção adversarial:

Observar Listar todos os fatos observáveis a partir da telemetria Sysmon (executável, linha de comando, processo-pai, usuário, *host*, *hashes*, diretório corrente, nível de integridade), sem interpretação.

Hipotetizar Formular duas hipóteses concorrentes e explícitas — uma maliciosa e uma benigna — para a atividade observada.

Evidência Citar a telemetria específica que sustenta ou contradiz cada hipótese, ancorando o raciocínio em fatos verificáveis.

Validação Cruzada Conferir a coerência entre a linha de comando, o processo-pai, o usuário, o diretório de trabalho e a atividade de rede, buscando confirmação ou contradição mútua entre os campos.

Classificar Emitir o veredito final (*malicious* ou *benign*) com escore de confiança, justificado pelos passos anteriores.

A sequência OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR cumpre uma função dupla. Como protocolo cognitivo, ela impõe um raciocínio deliberado e auditável, reduzindo a propensão do modelo a saltar diretamente para uma conclusão sugerida pelo conteúdo. Como anteparo de segurança, ela ancora a classificação na evidência primária: como o veredito decorre dos passos OBSERVAR e EVIDÊNCIA, uma instrução injetada que não corresponda a um fato observável não tem por onde influenciar o resultado. A capacidade de tratar texto injetado como dado é, assim, uma consequência estrutural do *framework*, e não uma regra avulsa. Note-se ainda que o passo VALIDAÇÃO CRUZADA interno ao raciocínio de cada caçador é distinto, em escopo, do mecanismo M2 da Seção 4.3.2: o primeiro é a corroboração *intra-agente* entre campos de uma mesma evidência; o segundo é a corroboração *interagente* de um achado por um validador independente. Os dois operam em níveis complementares de defesa.

4.5 Implementação como Plugin do Claude Code

A arquitetura descrita não é uma abstração de papel: ela é materializada como um *plugin* do Claude Code, ambiente de agente de linha de comando que expõe capacidades nativas de orquestração de subagentes. Esta seção descreve os três artefatos de extensão

que compõem o *plugin* — habilidades, comandos de barra e *hooks* — e o modo como o agente principal os combina para instanciar a hierarquia de três camadas. A noção de organizar agentes de LLM por arcabouços extensíveis e modulares é coerente com os *frameworks* de agentes contemporâneos [Guo et al. 2024]; o MAS-Hunt particulariza essa modularidade para o domínio da caça a ameaças sob governança.

4.5.0.0.1 Habilidades (*skills*).

As habilidades encapsulam o conhecimento metodológico e procedimental do sistema. São elas que codificam, por exemplo, o protocolo do Conselho de Governança (votação, quórum, roteamento por camada), o *framework* de construção de consultas ao Elastic Stack (seleção entre DSL, EQL e ES|QL conforme a tarefa) e os padrões de integração com inteligência externa. Uma habilidade não executa por si só; ela fornece ao agente o conhecimento de *como* agir corretamente quando uma tarefa do domínio é acionada, promovendo consistência metodológica entre execuções e entre agentes.

4.5.0.0.2 Comandos de barra (*slash commands*).

Os comandos de barra são os pontos de entrada que disparam fluxos completos. O comando que executa o experimento de detecção, por exemplo, implementa a arquitetura de governança como uma sucessão de cinco equipes (*teams*) consecutivas — Conselho de Governança, Gerência, Execução, Validação e Revisão Final do Conselho —, cada qual cumprindo sua fase, escrevendo seus artefatos em `.orchestration/` e sendo encerrada antes que a próxima inicie. Comandos distintos materializam as diferentes condições experimentais: o fluxo completo com governança (C1-full) e o fluxo de referência desprovido de governança (C2-naive), no qual nenhuma das equipes de validação ou dos mecanismos M1–M5 é instanciada. Os comandos são, portanto, a face executável das condições de ablação.

4.5.0.0.3 *Hooks*.

Os *hooks* são pontos de interceptação que acoplam verificações automáticas ao laço de execução do agente, atuando às fronteiras das chamadas de ferramenta e do encerramento de tarefas. No MAS-Hunt, os *hooks* são o substrato técnico de parte do endurecimento: eles podem acionar a sanitização de conteúdo (M3), registrar entradas na trilha de auditoria em formato compatível com o Elastic Common Schema, e impor portas de verificação que bloqueiam o encerramento de uma fase enquanto critérios de evidência não forem observavelmente satisfeitos. Por operarem na fronteira do laço do agente — e não no corpo do *prompt* —, os *hooks* aplicam controles que o próprio agente não pode contornar por meio de raciocínio manipulado.

A orquestração propriamente dita é responsabilidade do *agente principal*. Por meio das capacidades nativas do Claude Code, o agente principal cria equipes, instancia subagentes

(caçadores, gerentes, membros do Conselho), distribui-lhes tarefas e coleta seus resultados, percorrendo o *pipeline* de caça (Figura 2): *board* → *management* → caçadores por família LOLBin (em paralelo) → validação → *board*. Cada subagente recebe um contexto próprio e um conjunto restrito de ferramentas, e comunica-se exclusivamente pelo protocolo de mensagens tipadas da Seção 4.6. É a combinação entre habilidades (o que saber), comandos (o que disparar), *hooks* (o que verificar automaticamente) e a orquestração nativa de subagentes (quem faz o quê, e quando) que dá existência efetiva à arquitetura de três camadas.

4.6 Protocolos de Mensagens Tipadas e Governança

A comunicação entre camadas não é texto livre: ela é mediada por um *protocolo de mensagens tipadas* que torna cada transição verificável, assinada e roteável. Toda mensagem é um envelope com tipo declarado, identificador único, identificação do remetente (*sender_id*, *sender_role* e *sender_layer*), referência ao objetivo de caça governante (*hunt_goal_ref*), identificador de correlação para rastreabilidade (*correlation_id*), carga útil (*payload*) e assinatura. O protocolo impõe quatro garantias:

1. **Validação por esquema.** Cada tipo de mensagem possui um esquema JSON próprio; uma mensagem que não conforme ao esquema é rejeitada e desviada para uma fila de mensagens mortas (*dead-letter*).
2. **Assinatura criptográfica.** Cada mensagem é assinada por HMAC-SHA256 sobre sua forma canônica, de modo que a adulteração de conteúdo ou a falsificação de remetente seja detectável na recepção.
3. **Roteamento por adjacência de camada.** As mensagens classificam-se em descendentes (*HuntGoal*, *HuntTask*, *IntelTask*, *DetectionTask*), ascendentes (*HuntFinding*, *HuntReport*, *IntelFeed*, *ValidationVerdict*) e laterais (*DetectionRule*, *IntelReport*, *ValidationReport*). O protocolo verifica o sentido (uma diretiva descendente não pode partir de uma camada inferior) e adverte sobre violações de adjacência (saltos de mais de uma camada), reforçando a mediação por fronteira descrita na Seção 4.1.
4. **Fila de mensagens mortas.** Mensagens inválidas, não assinadas ou mal roteadas não são silenciosamente descartadas: são preservadas com o motivo da rejeição, alimentando o monitoramento comportamental (M4) e a auditoria.

Essa tipagem estrita realiza, no plano da comunicação, os princípios de uso de mensagens estruturadas, propagação de identificadores de correlação e registro de decisões de agente que caracterizam a coordenação multiagente disciplinada [Qiu et al. 2024]. O roteamento mediado por uma autoridade que delega e consolida é, ainda, a contraparte técnica da organização hierárquica de agentes [Liu et al. 2024].

4.6.1 Por que a Orquestração Real Difere de uma Chamada LLM Isolada

O ponto central deste capítulo é que a contribuição do MAS-Hunt reside na orquestração multiagente governada, e não em *scripts* de detecção ou em uma chamada bem elaborada a um modelo de linguagem. Uma chamada isolada a um LLM, ainda que com um *prompt* sofisticado, é uma operação sem estado, sem mediação e sem contenção: ela recebe um contexto, produz uma saída e não impõe qualquer disciplina sobre como essa saída é formada, validada ou contida. Cinco diferenças estruturais separam a orquestração real do MAS-Hunt desse regime.

1. **Separação de poderes entre agentes.** Em uma chamada isolada, um único agente concentra descoberta, julgamento e veredito. No MAS-Hunt, essas funções são distribuídas por camadas distintas — caçadores descobrem, validadores corroboram, o Conselho decide — de modo que nenhum agente isolado determina o resultado. O comprometimento de um agente não compromete o sistema.
2. **Estado, memória e auditoria persistentes.** A orquestração mantém estado entre fases — objetivos, planos, achados, vereditos, votos — em uma trilha de auditoria imutável. Uma chamada isolada não tem memória do que ocorreu antes nem registro do que ocorreu depois; não há, nela, base para reprodutibilidade ou análise post-mortem.
3. **Mediação de comunicação por protocolo tipado.** O conteúdo que trafega entre agentes é validado por esquema, assinado e roteado por adjacência de camada. Em uma chamada isolada, todo conteúdo — inclusive conteúdo hostil — entra indiscriminadamente no mesmo canal do *prompt*, sem fronteira entre dado e instrução.
4. **Defesa ativa contra manipulação do próprio agente.** Os mecanismos M1–M5 monitoram, detectam e contêm o comprometimento dos agentes em tempo de execução. Uma chamada isolada não dispõe de monitoramento comportamental nem de quarentena; se for envenenada ou injetada, ela simplesmente produz a saída manipulada.
5. **Governança deliberativa por quórum.** As decisões estratégicas emergem de um colegiado que vota e registra, e não de um oráculo único. Essa deliberação é o que confere ao sistema robustez a participantes defeituosos, propriedade ausente por construção em uma chamada isolada.

Em síntese, o MAS-Hunt não é um classificador de eventos com um *prompt* melhor; é um sistema de governança que organiza, valida e contém o comportamento de múltiplos agentes ao redor da tarefa de caça. As condições experimentais materializam exatamente essa distinção: C2-naive aproxima-se de um conjunto de agentes sem governança — o regime que esta seção contrasta —, enquanto C1-full aplica toda a pilha de mediação e contenção. A comparação entre essas condições, e a ablação intermediária C1-nohardening,

é o instrumento empírico pelo qual a dissertação isola e mensura o valor da governança multiagente, objeto dos Capítulos 5 e seguintes.

5 Metodologia Experimental

Este capítulo descreve o desenho experimental empregado para avaliar o MAS-Hunt sob as duas afirmações centrais deste trabalho: a efetividade de detecção sobre telemetria empresarial realista e a resiliência da própria camada de agentes contra a manipulação adversarial da informação que eles analisam. A avaliação foi concebida como um experimento controlado e plenamente reproduzível, no qual todas as condições comparadas são pontuadas sobre execuções idênticas, janelas de detecção idênticas e o mesmo *ground truth*, de modo a isolar a contribuição da arquitetura de governança proposta dos demais fatores. As seções seguintes apresentam, nesta ordem: o ambiente de laboratório instrumentado (Seção 5.1); o corpus de ataque e o critério de verdade de referência (Seção 5.2); o desenho adversarial de injeção de logs (Seção 5.3); as quatro condições comparadas (Seção 5.4); o *pipeline* de medição, incluindo a extração de telemetria, o gatilho de saúde (*health-gate*) e a reprodutibilidade (Seção 5.5); as métricas e a análise estatística (Seção 5.6); e, por fim, as limitações e ameaças à validade (Seção 5.7).

Cumpre destacar, de saída, uma decisão metodológica que governa todo o capítulo: **nenhum resultado numérico é apresentado aqui**. Esta seção documenta exclusivamente o *desenho* do experimento e os procedimentos de medição; os valores observados de precisão, revocação, F2, taxa de falsos positivos e taxa de sucesso de ataque são reportados e discutidos no Capítulo 6.

5.1 Ambiente de Laboratório Instrumentado

5.1.1 Topologia de domínio Active Directory

Os experimentos foram conduzidos sobre um ambiente virtualizado de *Active Directory* hospedado em um único hipervisor KVM. As máquinas virtuais Windows foram provisionadas e gerenciadas por meio da pilha Dockur/QEMU, que encapsula instâncias QEMU/KVM em contêineres, simplificando o ciclo de provisionamento, restauração e instrumentação dos convidados Windows. O domínio foi composto por três hospedeiros:

- **dc01** — um controlador de domínio executando Windows Server 2025, responsável pelos serviços de diretório, autenticação e políticas de grupo do domínio;
- **ws01** e **ws02** — duas estações de trabalho Windows 11 ingressadas no mesmo domínio *Active Directory*, representando os *endpoints* de usuário.

Essa topologia reproduz deliberadamente a superfície de movimentação lateral de uma pequena rede corporativa — com autenticação de domínio, relações de confiança e tráfego entre hospedeiros — em vez do *sandbox* estéril de um único hospedeiro contra o

qual detectores baseados em aprendizado de máquina costumam ser treinados e avaliados [Ucci, Aniello e Baldoni 2019]. A presença de mais de uma estação ingressada no domínio é o que permite que técnicas de movimentação lateral e de execução remota (por exemplo, via WinRM) gerem cadeias de processos distribuídas entre hospedeiros, exigindo correlação entre máquinas — uma das capacidades que o MAS-Hunt visa exercitar.

5.1.2 Instrumentação de telemetria

Cada *endpoint* foi instrumentado para visibilidade comportamental de alta fidelidade por meio de três camadas complementares de coleta:

1. **Sysmon com configuração ajustada para LOLBins.** O System Monitor (Sysmon) foi implantado com uma configuração afinada para o abuso de binários nativos do sistema operacional (*living-off-the-land binaries*, LOLBins). A configuração captura, em especial, eventos de criação de processo (*Event ID* 1, doravante EID1) com os respectivos argumentos de linha de comando, a linhagem pai-filho de processos, conexões de rede, carregamento de imagens (*image load*) e modificações de registro. O ajuste para LOLBins é central: como o vetor de ataque deste trabalho são binários legítimos do Windows (*bitsadmin.exe*, *certutil.exe*, *regsvr32.exe*, *rundll32.exe*, *mshta.exe*, *wmic.exe*, entre outros), o sinal forense discriminativo reside menos na identidade do binário e mais nos argumentos de linha de comando e na relação pai-filho de processos, que a configuração privilegia.
2. **Elastic Agent com a integração Elastic Defend.** Em paralelo ao Sysmon, o Elastic Agent — gerenciado centralmente pelo Fleet — foi implantado com a integração Elastic Defend, fornecendo telemetria de proteção de *endpoint* (EDR) e enriquecendo a cobertura de eventos de processo, rede e arquivo.
3. **Políticas avançadas de auditoria do Windows.** As políticas de auditoria avançada do Windows foram habilitadas para complementar a cobertura com eventos de segurança nativos (*Security event log*), em especial os relacionados a autenticação, criação de processos e uso de privilégios.

5.1.3 Plataforma analítica: Elastic 9.3.4

Toda a telemetria das três camadas foi enviada a uma implantação do Elastic na versão 9.3.4, composta por:

- **Elasticsearch** — armazenamento e indexação dos eventos, com os eventos de Sysmon indexados sob o fluxo de dados (*data stream*) `logs-windows.sysmon_operational-default`;
- **Kibana** — interface de análise, visualização e consulta;

- **Fleet** — gerenciamento centralizado dos Elastic Agents e das políticas de integração implantadas nos três hospedeiros.

A implantação Elastic serviu simultaneamente como *fonte única de verdade* para a telemetria e como *campo de caça analítico* comum a todas as condições avaliadas. Em particular, a quarta condição experimental (a linha de base C3-Elastic, definida na Seção 5.4) emprega exatamente o mesmo *deployment* e o conjunto padrão de regras de detecção do Elastic Security, garantindo que a comparação entre a camada agêntica proposta e a linha de base SIEM seja feita sobre a mesma telemetria, sem qualquer vantagem artificial de coleta.

5.2 Corpus de Ataque e Verdade de Referência

5.2.1 Os 185 *playbooks* em nove famílias

O corpus de detecção é composto por **185 *playbooks*** de ataque baseados em LOLBins, distribuídos por **nove famílias de técnica** correspondentes aos binários nativos mais comumente abusados em campanhas reais:

bitsadmin — abuso do serviço de transferência inteligente em segundo plano (BITS) para *download* e execução furtiva de *payloads*;

certutil — uso indevido do utilitário de certificados para *download*, codificação e decodificação de *payloads*;

credential-access — técnicas de acesso a credenciais (por exemplo, extração de segredos e abuso de processos sensíveis);

lateral-movement — execução remota e movimentação entre hospedeiros do domínio;

mshta — execução de aplicações HTML maliciosas via `mshta.exe`;

privilege-escalation — técnicas de elevação de privilégios;

regsvr32 — execução de *scriptlets* e DLLs via `regsvr32.exe` (*Squiblydoo* e variantes);

rundll32 — execução de código por meio de `rundll32.exe`;

wmic — abuso da Instrumentação de Gerenciamento do Windows (WMI) para execução e reconhecimento.

Cada família contém, deliberadamente, ***playbooks* maliciosos misturados a variantes benignas** que exercitam exatamente os mesmos binários para fins administrativos legítimos. Esse balanceamento malicioso vs. benigno é uma decisão de desenho destinada a estressar a *precisão* de cada condição, em vez de premiar o alarme indiscriminado: um

detector que simplesmente sinalizasse toda ocorrência de `certutil.exe` ou `rundll32.exe` como maliciosa obteria revocação trivial às custas de uma taxa de falsos positivos inaceitável em ambientes operacionais de baixa taxa-base. A discriminação entre o uso benigno e o malicioso do mesmo binário é precisamente o problema de detecção que se deseja medir.

Cada *playbook* é executado em um dos três hospedeiros do domínio, produzindo uma **janela de execução determinística** $[t_{\text{início}}, t_{\text{fim}}]$ registrada contra o hospedeiro de origem. Essa janela é a unidade fundamental de medição de todo o experimento, conforme detalhado na Seção 5.5.

5.2.2 Critério de verdade de referência (*ground truth*)

A verdade de referência foi atribuída **por *playbook***, na granularidade de uma janela de execução. Um *playbook* é rotulado como *malicioso* se, e somente se, o seu caminho de arquivo não denotar uma variante benigna — isto é, se o caminho não contiver o prefixo `benign-` nem o segmento `/benign/`. Formalmente, define-se a função determinística:

$$é_malicioso(p) = (\text{benign-} \notin p) \wedge (/benign/ \notin p), \quad (5.1)$$

onde p é o caminho relativo do *playbook*. Essa regra de rotulação é determinística, auditável e independente de qualquer juízo de modelo de linguagem.

Uma salvaguarda metodológica essencial acompanha esse critério: o caminho do *playbook* — e, portanto, o rótulo — é mantido em um **arquivo lateral** (`ground_truth.jsonl`) que *nunca* é exposto aos agentes de linguagem durante a classificação. Desse modo, nenhuma condição pode inferir o rótulo a partir do nome do arquivo (*anti-cheat*); os agentes classificam exclusivamente a partir da telemetria observável da janela. Durante a pontuação, cada janela de *playbook* é reconciliada com o seu rótulo por meio de uma chave de junção, e a predição emitida pela condição sob teste é então comparada ao rótulo verdadeiro.

5.3 Desenho Adversarial de Injeção de Logs

A avaliação de resiliência introduz ***payloads* adversariais de injeção de logs** — conteúdo controlado pelo atacante, embutido no fluxo de telemetria e concebido para subverter o raciocínio dos agentes. Esse vetor materializa, no domínio de caça a ameaças, as ameaças de injeção indireta de *prompt* [Zhan et al. 2024], de envenenamento de memória [Chen et al. 2024] e de amplificação de mau funcionamento [Zhang et al. 2025] que motivam este trabalho. O desenho adversarial cruza dois fatores ortogonais: o *modo de injeção* e o *nível de sofisticação*.

5.3.1 Quatro modos de injeção

none (nenhum — controle) Condição de controle, sem qualquer conteúdo adversarial injetado. Mede o desempenho de detecção em telemetria limpa e serve de referência para quantificar a degradação imposta pelos demais modos.

pre-burst (rajada prévia) Uma rajada concentrada de *payloads* adversariais é injetada *antes* da execução do corpus de ataque, contaminando o contexto de logs que os agentes encontrarão.

per-attack-burst (rajada por ataque) Uma rajada de *payloads* envolve *cada* execução de *playbook*, aproximando temporalmente o conteúdo adversarial dos eventos genuínos da janela sob análise.

interleaved (intercalado) *Payloads* são continuamente intercalados em meio à atividade benigna e maliciosa, representando o cenário mais difuso e persistente de contaminação.

5.3.2 Três níveis de sofisticação

overt (explícito) *Payloads* que carregam, de forma claramente visível, texto de sobreposição de instrução (*instruction override*) — por exemplo, marcadores como “ignore a análise anterior” diretamente legíveis nos campos de log.

encoded (codificado) *Payloads* que ofuscam a mesma intenção por meio de esquemas de codificação (por exemplo, Base64 ou transformações análogas), exigindo do agente a decodificação implícita para que a tentativa de subversão se manifeste.

steganographic (esteganográfico) *Payloads* que ocultam a instrução maliciosa dentro de campos de log de aparência legítima, representando o disfarce mais sutil entre os três níveis.

5.3.3 Estrutura fatorial e *payloads*

A combinação fatorial dos dois fatores produz a condição de controle (modo *none*) mais nove condições adversariais (3 modos $\times$ 3 níveis, excluindo o controle sem injeção do cruzamento de níveis), totalizando **dez condições de execução**. Cada uma das dez condições executa o corpus completo de 185 *playbooks* distribuído entre os três hospedeiros, de modo que todas as condições são pontuadas sobre o mesmo conjunto de janelas. Esse desenho permite investigar, de forma sistemática, duas hipóteses: (i) se a contaminação do contexto de logs degrada a detecção — o objetivo de falso negativo dos *payloads* injetados — e (ii) se o aumento do disfarce, do explícito ao esteganográfico, erode progressivamente as defesas dos agentes.

Cada *payload* adversarial é descrito em um manifesto, identificado por um *injection_id* único, e caracterizado por: a técnica de injeção, a categoria de objetivo (por exemplo, induzir falso negativo), o tipo de log alvo (Sysmon, PowerShell, *Application* ou *Security*), o texto do *payload* e o nível de evasão (*overt*, *encoded* ou *steganographic*). O instante de injeção de cada *payload* ($[t_{inj, início}, t_{inj, fim}]$) é registrado por hospedeiro, o que permite associar a janela temporal de contaminação às janelas de execução dos *playbooks* durante a análise.

5.4 Condições Comparadas

Quatro condições de detecção foram pontuadas sobre as mesmas execuções, as mesmas janelas e o mesmo *ground truth*. Todas as condições emitem uma decisão binária malicioso/benigno *por janela*, que é reconciliada com a verdade de referência por um *único script* de pontuação determinístico e não baseado em inteligência artificial — desacoplando a descoberta flexível (a cargo dos agentes ou regras) da avaliação determinística (a cargo do pontuador). As quatro condições são as seguintes.

C1-full O sistema MAS-Hunt completo, compreendendo a hierarquia de governança de três camadas (Conselho de Governança → Gerentes → Trabalhadores/Caçadores) e o complemento integral de mecanismos de proteção M1–M5 (Integridade de Memória, Validação Cruzada, Resistência a Injeção, Monitoramento Comportamental e Quarentena), em conjunto com o *framework* de análise M3, que conduz o laço de raciocínio OBSERVAR → HIPOTETIZAR → EVIDÊNCIA → VALIDAÇÃO-CRUZADA → CLASSIFICAR. Esta é a condição que materializa a contribuição central do trabalho.

C1-nohardening Uma ablação de C1-full na qual os mecanismos de endurecimento e governança (M1, M2, M4 e M5) são desativados, preservando apenas o *framework* de análise M3. Esta condição isola a contribuição marginal da camada de proteção, mantendo constantes os agentes subjacentes e o laço de raciocínio.

C2-naive Um classificador de modelo de linguagem de passagem única, que observa as mesmas janelas de telemetria, mas não aplica governança, nem validação cruzada, nem defesas contra injeção. Representa a linha de base agêntica não governada — um único agente classificando cada janela de forma isolada.

C3-Elastic O conjunto padrão de regras de detecção do Elastic Security, representando a linha de base SIEM “de prateleira” que um profissional obteria sem qualquer camada agêntica. As predições de C3-Elastic são derivadas das contagens de alertas geradas pelas regras padrão dentro de cada janela de *playbook*.

A progressão C1-full → C1-nohardening → C2-naive constitui uma **ablação graduada** que atribui o desempenho, respectivamente, à arquitetura de governança, à camada

agêntica e ao raciocínio de modelo de linguagem isolado; C3-Elastic ancora a comparação na linha de base SIEM. A literatura de caça a ameaças assistida por aprendizado de máquina [Ali e Kostakos 2023] e de coordenação multiagente [Mahmud, Goldfajn e Zilberstein 2025] motiva essa estrutura de comparação graduada, na qual cada degrau acrescenta exatamente um componente arquitetural ao anterior.

5.5 Pipeline de Medição

O *pipeline* de medição converte cada execução de corpus em um conjunto de métricas reconciliadas com o *ground truth*. Ele é composto por três estágios — *preparação* (extração de janelas de telemetria), *classificação* (a cargo da condição sob teste) e *pontuação* (reconciliação determinística) — precedidos por um gatilho de saúde de telemetria (*health-gate*) que condiciona a própria execução do corpus. Esta seção descreve cada estágio.

5.5.1 Janelas por *playbook* e extração de telemetria

A unidade de medição é a **janela por *playbook***. Para cada um dos 185 *playbooks* de uma execução, atribui-se um identificador de janela `window_id` (na forma `w0001`, `w0002`, ...), e extrai-se o hospedeiro de origem, o intervalo $[t_{\text{início}}, t_{\text{fim}}]$ e os eventos de telemetria observados nessa janela. A extração consulta o `data stream logs-windows.sysmon_operational-default` do Elasticsearch, filtrando por intervalo temporal e por hospedeiro (`host.name` ou `host.hostname`), e compacta cada evento a um registro enxuto contendo o *timestamp*, o *Event ID*, a ação, o processo, o PID, a linha de comando, o processo pai e sua linha de comando, e o usuário. O isolamento entre janelas é garantido pela *disjunção temporal* dos intervalos $[t_{\text{início}}, t_{\text{fim}}]$, e não por índices separados.

O estágio de preparação produz dois artefatos: (i) `windows.jsonl`, com um registro por *playbook* contendo as janelas e os eventos compactados (*sem* o caminho do *playbook*, por *anti-cheat*); e (ii) o arquivo lateral `ground_truth.jsonl`, que mapeia cada `window_id` ao caminho do *playbook* e é usado exclusivamente pelo pontuador, nunca exposto ao agente classificador. A chave de junção entre predições, janelas e verdade de referência é o `window_id`.

5.5.2 Extração *prepare-v2*: priorização de EID1

A versão inicial da extração (*prepare*) recolhia, por janela, os primeiros eventos em ordem cronológica até um teto fixo (60 eventos). Constatou-se que esse critério cronológico plano permitia que o ruído de eventos de registro (EID 12/13), dominante no volume de telemetria, deslocasse os eventos de *criação de processo* (EID1) — justamente os mais discriminativos para a detecção de LOLBins — para fora da janela amostrada. Adotou-se, portanto, uma extração aprimorada, denominada *prepare-v2*, que preserva identicamente o esquema de saída (registros indexados por `window_id`, com rótulos retidos no arquivo

lateral, de modo que a pontuação permanece inalterada), acrescentando o campo `n_eid1` e aplicando três mudanças:

1. **Priorização de EID1.** Todos os eventos de criação de processo (EID1) presentes na janela ampliada são recuperados (até um teto de candidatos) e *sempre* incluídos na amostra;
2. **Preenchimento subordinado.** Os demais eventos (não-EID1) são então acrescentados em ordem cronológica até o teto de eventos por janela (elevado para 120);
3. **Ampliação do *padding*.** O *padding* temporal da janela é ampliado para ± 120 s (ante os ± 60 s da versão inicial), a fim de absorver a latência de ingestão dos agentes/Sysmon e o desvio de relógio dos convidados Windows.

Em execuções ricas em sinal, essa priorização recupera a cadeia de criação de processos completa de um ataque conduzido remotamente (por exemplo, a cadeia `winrshost.exe` $\rightarrow$ `cmd.exe` $\rightarrow$ `powershell.exe` de uma execução via WinRM), que a extração cronológica plana descartava. A motivação de desenho e a sua relação com a lacuna de telemetria de EID1 são discutidas na Seção 5.7.

5.5.3 O gatilho de saúde de telemetria (*health-gate*)

Um dos achados metodológicos mais importantes deste trabalho diz respeito à *prontidão* da instrumentação após a restauração de um *snapshot* limpo. As primeiras execuções enviaram telemetria *quebrada*: os *playbooks* eram executados imediatamente após a restauração da máquina virtual, antes que os Elastic Agents e o Sysmon das estações de trabalho tivessem concluído o *check-in* com o Fleet e iniciado o envio de eventos. O resultado era assimétrico e silencioso — por exemplo, uma estação enviando zero eventos, outra enviando poucas centenas e a criação de processos (EID1) dos *playbooks* maliciosos ausente do Elasticsearch. Uma espera fixa (*sleep* cego) não constitui um sinal confiável de prontidão, porque disputa uma corrida com tempos de inicialização variáveis.

Para sanar esse problema, introduziu-se um **gatilho de saúde de telemetria** (*telemetry health-gate*) que substitui a espera cega. O gatilho consulta o Elasticsearch repetidamente e *bloqueia a execução do corpus* até que os três hospedeiros (dc01, ws01 e ws02) estejam comprovadamente ativos em ambas as camadas das quais o orquestrador depende. Um hospedeiro é considerado *saudável*, a cada sondagem, se, e somente se:

1. houver enviado, desde o início do gatilho, um número mínimo de documentos *frescos* de criação de processo (EID1) ao Elasticsearch; e
2. o seu *heartbeat* de WinRM responder com sucesso (porta alcançável e capaz de executar comandos).

5.5.3.0.1 Por que o gatilho é necessário.

A instrumentação restaurada a partir de um *snapshot* precisa de *tempo para estabilizar*: os agentes e o Sysmon levam um intervalo variável para iniciar o envio após a restauração. O gatilho de saúde torna a prontidão um sinal *observável e verificado* — em vez de uma suposição temporal — e, com isso, garante que toda execução de *playbook* ocorra somente quando a cadeia completa WinRM alcançável → processo gerado → Sysmon EID1 → Fleet → Elasticsearch estiver comprovadamente funcional. Para forçar esse sinal ativamente, o gatilho dispara, a cada sondagem, alguns processos efêmeros via WinRM em cada hospedeiro ainda não saudável; se a cadeia de instrumentação estiver viva, esses processos afloram como EID1 fresco em poucos segundos, transformando o gatilho em uma sonda *ponta-a-ponta* de toda a cadeia que antes falhava silenciosamente.

Duas decisões de robustez merecem registro. Primeiro, a frescura dos eventos é medida *no espaço de timestamps do Elasticsearch*, e não contra o relógio do hospedeiro do laboratório: observou-se que os relógios dos convidados Windows operam adiantados em relação ao relógio do hospedeiro, de modo que uma comparação ingênua “@timestamp ≥ agora – 3 min” contra o relógio do hospedeiro encontraria zero eventos em um hospedeiro saudável. Por isso, no início do gatilho registra-se o *timestamp* máximo corrente de EID1 de cada hospedeiro (congelado, pois as VMs estão desligadas/em restauração) e contam-se apenas os documentos EID1 estritamente posteriores a essa marca. Segundo, exigir o *heartbeat* de WinRM como parte da saúde fecha, sem custo adicional, a lacuna de um hospedeiro que envia Sysmon mas tem o WinRM indisponível — caso em que os *playbooks* roteados para ele falhariam.

5.5.4 Reprodutibilidade: restauração limpa e *snapshots*

A reprodutibilidade é tratada como contribuição de primeira classe, e não como um adendo: todo o laboratório de avaliação e o arsenal de medição são re-executáveis de ponta a ponta a partir de *snapshots* versionados. A topologia de três VMs, o corpus de *playbooks* (185 *playbooks*), a condição de controle e as nove condições adversariais, o validador determinístico e o *pipeline* de pontuação por *bootstrap* são todos capturados como *snapshots* de linha de base instrumentados.

Cada condição é executada a partir de uma **restauração limpa**: as três VMs são restauradas a um estado de linha de base conhecido entre condições, o que assegura o isolamento estrutural entre execuções (a contaminação adversarial de uma condição não vaza para a seguinte). Para conter um pico de carga de memória observado durante a inicialização simultânea das três VMs, adotou-se um esquema de **inicialização escalonada** (*staggered boot*): cada VM é iniciada e submetida ao gatilho de saúde individualmente, em sequência (dc01, depois ws01, depois ws02), antes que a próxima seja iniciada, eliminando a tempestade de inicialização simultânea. Esse procedimento foi essencial para a estabilidade

do hospedeiro do laboratório, que opera com folga estreita de memória ao executar três VMs Windows mais a pilha Elastic completa.

A combinação de *snapshots* versionados, validador determinístico e comparação baseada em *bootstrap* confere às medições uma garantia de reprodutibilidade *verificável* em vez de apenas *afirmada*: um terceiro de posse dos *snapshots* pode reconstituir o experimento sem reconfiguração manual — propriedade que, ela própria, é testada empiricamente no Capítulo 6 por meio de uma re-execução limpa a partir da linha de base instrumentada.

5.6 Métricas e Análise Estatística

5.6.1 Métricas de detecção

O desempenho de detecção é avaliado por *playbook*, a partir das contagens de matriz de confusão de verdadeiros positivos (*VP*), falsos positivos (*FP*), falsos negativos (*FN*) e verdadeiros negativos (*VN*), nas quais um *positivo* denota uma janela classificada como maliciosa. Como, em caça a ameaças, o custo operacional de uma intrusão não detectada supera o de um alerta supérfluo, ao passo que uma taxa de falsos positivos descontrolada inviabiliza qualquer detector em ambientes de baixa taxa-base [Dolan-Gavitt 2025], reporta-se um equilíbrio entre as duas grandezas ponderado pela revocação.

- **Precisão** — a fração de janelas sinalizadas como maliciosas que eram, de fato, maliciosas:

$$\text{Precisão} = \frac{VP}{VP + FP}. \quad (5.2)$$

- **Revocação** (*recall*, equivalente à Taxa de Verdadeiros Positivos) — a fração de janelas maliciosas corretamente identificadas:

$$\text{Revocação} = \frac{VP}{VP + FN}. \quad (5.3)$$

- **Score F2** — a média harmônica ponderada de precisão e revocação, com $\beta = 2$, na qual a revocação é ponderada quatro vezes mais que a precisão, refletindo o custo assimétrico das detecções perdidas:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{(\beta^2 \cdot \text{Precisão}) + \text{Revocação}}, \quad F_2 = 5 \cdot \frac{\text{Precisão} \cdot \text{Revocação}}{(4 \cdot \text{Precisão}) + \text{Revocação}}. \quad (5.4)$$

- **Taxa de Falsos Positivos (FPR)** — a taxa de alarmes falsos entre a atividade benigna, reportada de duas formas complementares: por janela benigna, e normalizada por 100 000 eventos de Sysmon observados na janela do corpus, de modo a permitir a comparação entre condições a um volume comum de telemetria:

$$\text{FPR} = \frac{FP}{FP + VN}. \quad (5.5)$$

O cômputo das métricas é feito por um *script* determinístico, idêntico para todas as quatro condições. Predições ausentes (janelas para as quais a condição não emitiu decisão) são tratadas como benignas, opção conservadora que penaliza a revocação. As métricas são, ainda, decompostas *por família de LOLBin* (a primeira componente do caminho do *playbook*), permitindo a análise diferencial por técnica.

5.6.2 Métricas de resiliência adversarial

A resiliência sob as condições adversariais da Seção 5.3 é quantificada por duas métricas complementares:

- **Taxa de Sucesso de Ataque (ASR)** — a fração de tentativas de injeção em que o *payload* adversarial atinge o seu objetivo de forçar uma classificação incorreta:

$$ASR = \frac{N_{\text{classificadas_incorretamente}}}{N}. \quad (5.6)$$

- **Taxa Líquida de Sucesso de Ataque (NASR)** — contabiliza apenas o desfecho estritamente pior: tentativas que foram *ao mesmo tempo* classificadas incorretamente e nas quais a injeção *não foi detectada* como adversarial. Por construção, $NASR \leq ASR$, pois uma classificação incorreta que a camada de governança simultaneamente sinaliza como injeção detectada não constitui um comprometimento silencioso:

$$NASR = \frac{N_{\text{incorreta} \wedge \text{injeção_não_detectada}}}{N}. \quad (5.7)$$

Uma condição resiliente exhibe ASR e NASR mais baixas. O contraste entre C1-full e a linha de base não governada C2-naive mede o valor protetor da camada de governança contra os modos de falha de propagação infecciosa e de corrupção de raciocínio documentados na literatura de segurança de sistemas multiagente [Huang et al. 2025].

5.6.3 Análise estatística por *bootstrap*

Para assegurar que as diferenças observadas não sejam artefatos de amostragem, cada estimativa pontual reportada é acompanhada de um **intervalo de confiança (IC) de 95% obtido por *bootstrap* não-paramétrico**. Para cada métrica, extraem-se $B = 10\,000$ reamostras com reposição do vetor de desfechos por *playbook* de cada condição (braço), recomputa-se a métrica em cada reamostra e reportam-se os percentis de 2,5% e 97,5% da distribuição *bootstrap* como os limites inferior e superior do intervalo de 95% (método percentil). Esse procedimento não faz qualquer suposição paramétrica sobre a distribuição amostral e é aplicado de forma idêntica à Precisão, à Revocação, ao F2 e às métricas de resiliência.

Uma diferença entre condições só é tratada como estatisticamente relevante quando os intervalos de confiança correspondentes a sustentam. De modo análogo, a reprodutibilidade de cada métrica entre execuções independentes é avaliada por um **critério de**

sobreposição de intervalos: uma métrica é considerada estatisticamente reprodutível quando a estimativa pontual de cada execução recai dentro do intervalo de confiança da outra e os dois intervalos se sobrepõem.

5.7 Limitações e Ameaças à Validade

Em conformidade com a postura de integridade adotada neste trabalho, registram-se de forma explícita as limitações do desenho experimental e as ameaças à validade das conclusões.

5.7.1 A lacuna de telemetria de EID1

A limitação mais significativa é a **lacuna de telemetria de criação de processo (EID1)**. Na configuração de Sysmon empregada, a esmagadora maioria dos documentos corresponde a eventos de registro (EID 12/13); em execuções *pobres em sinal* (notadamente as condições de controle e *pre-burst*), as criações de processo (EID1) dos LOLBins são capturadas de forma esparsa, e parte das cadeias de processo dos *playbooks* maliciosos simplesmente não chega a ser registrada pelo Sysmon. Como o sinal forense discriminativo de um LOLBin reside, em grande medida, na criação do processo e em seus argumentos de linha de comando, essa lacuna impõe um *teto* à revocação alcançável, *independentemente* da condição — nenhuma quantidade de raciocínio recupera um evento que nunca foi gravado.

Esta dissertação enfrenta essa limitação de duas maneiras, ambas mitigadoras e não corretivas. Primeiro, a extração **prepare-v2** (Seção 5.5.2) *prioriza* EID1 na amostragem por janela, maximizando a captura de toda a criação de processo que de fato exista na telemetria; em execuções ricas em sinal, isso recupera a cadeia de ataque completa. Segundo, o gatilho de saúde (Seção 5.5.3) elimina a *causa instrumental* de parte da lacuna — a ausência de EID1 por instrumentação ainda não estabilizada após a restauração — ao exigir o envio verificado de EID1 fresco pelos três hospedeiros antes de qualquer execução. Cumpre, porém, ser explícito quanto à fronteira entre as duas: o gatilho e a **prepare-v2** sanam a lacuna *instrumental* (eventos que existiriam, mas não eram coletados a tempo ou eram deslocados da amostra), mas *não* sanam a lacuna *estrutural* (eventos que o Sysmon, na configuração dada, nunca chegou a logar). Em consequência, o ganho de C1-full sobre as linhas de base deve ser interpretado primordialmente como decorrente de *raciocínio contextual e correlação entre hospedeiros sob escassez de sinal*, e não como ganho de revocação bruta sobre eventos inexistentes.

5.7.2 Outras ameaças à validade

5.7.2.0.1 Validade interna.

A medição de resiliência opera no nível da *tentativa* (*trial-level*): a ASR e a NASR medem se o agente classificou incorretamente sob injeção, mas o corpus não dispõe de rótulos de contaminação *por evento*. Não se afirma, portanto, qual evento individual foi corrompido, apenas o desfecho da janela sob contaminação. Adicionalmente, a regra de *ground truth* baseada no caminho do arquivo (Equação 5.1) é determinística e auditável, mas pressupõe que a curadoria do corpus rotulou corretamente cada *playbook*.

5.7.2.0.2 Validade externa.

O ambiente é um domínio *Active Directory* de pequena escala (um controlador de domínio e duas estações), o que reproduz a superfície de movimentação lateral de uma pequena empresa, mas não a escala, a heterogeneidade nem o volume de *background* de uma rede empresarial de grande porte. O corpus de 185 *playbooks* cobre nove famílias de LOLBin, porém não esgota o universo de técnicas de *living-off-the-land*, e a distribuição malicioso/benigno do corpus não pretende refletir a taxa-base operacional real.

5.7.2.0.3 Validade de construto.

A linha de base C2-naive é definida como um classificador de passagem única; deve-se assegurar, na execução, que ela seja de fato um passe genuíno de modelo de linguagem (paridade com C1-full), e não uma heurística determinística, sob pena de a comparação medir algo distinto do pretendido. Esse cuidado é registrado aqui como ameaça de construto a ser controlada na fase de execução e auditada nos artefatos (Capítulo 6).

5.7.2.0.4 Estabilidade do laboratório.

A instância QEMU de uma das estações (ws02) apresentou instabilidade intermitente (asserção do monitor QEMU seguida de saída do contêiner) logo após a inicialização, sob a pressão de memória do hospedeiro. A inicialização escalonada, o *watchdog* de reinício e o re-tentar do gatilho mitigam essa instabilidade na fase de *boot*; uma falha *durante* a execução, contudo, ainda poderia perder os *playbooks* roteados para aquele hospedeiro naquela execução. Como salvaguarda, uma verificação de validade de telemetria, executada após cada condição, sinaliza qualquer execução com um hospedeiro ausente para re-execução dirigida.

Em síntese, o desenho experimental descrito neste capítulo isola a contribuição da arquitetura de governança por meio de condições pontuadas sobre janelas e verdade de referência idênticas, sob um *pipeline* de medição determinístico e reprodutível, com as suas limitações

— sobretudo a lacuna de telemetria de EID1 — documentadas de forma transparente. Os resultados quantitativos decorrentes deste desenho são apresentados no Capítulo [6](#).

6 Resultados

Este capítulo apresenta os resultados quantitativos da matriz experimental v2 do MAS-Hunt, coletada após a correção do gatilho de saúde de telemetria (*health-gate*) e da extração `prepare-v2` descritas no Capítulo 5. A apresentação é deliberadamente factual: os números são reportados com seus intervalos de confiança e contrastados entre as quatro condições, reservando-se a interpretação aprofundada e a discussão de suas implicações para o Capítulo 7. Em conformidade com a postura de integridade adotada ao longo do trabalho, os achados são relatados exatamente como observados, inclusive aqueles que *não* corroboram as expectativas iniciais.

6.1 Conjunto de Dados Analisado e Recorte Inferencial

A análise inferencial concentra-se nas **seis execuções adversariais com sinal efetivo de ataque**: os modos de injeção *per-attack-burst* (rajada por ataque) e *interleaved* (intercalado), cruzados com os três níveis de sofisticação *overt* (explícito), *encoded* (codificado) e *steganographic* (esteganográfico). Esse recorte responde a uma decisão metodológica documentada na Seção 5.7: as quatro execuções restantes (o controle sem injeção e as três variantes *pre-burst*) foram *pobres em sinal* de criação de processo (EID1) e, portanto, excluídas da matriz principal, sob pena de a escassez instrumental de telemetria mascarar o desempenho de detecção que se pretende medir. A exclusão é transparente e preserva a comparabilidade: todas as quatro condições são pontuadas sobre exatamente as mesmas seis execuções e as mesmas janelas.

Em cada uma das seis execuções, foram avaliadas 182 janelas — uma por *playbook* que produziu telemetria de criação de processo —, totalizando **1.092 janelas** por condição, das quais **894 são maliciosas** e **198 são benignas** segundo o critério de verdade de referência da Equação 5.1. As quatro condições foram pontuadas pelo *mesmo script* determinístico, não baseado em inteligência artificial, garantindo que qualquer diferença observada decorra da condição, e não do procedimento de avaliação. Os intervalos de confiança (IC) de 95% reportados foram obtidos por *bootstrap* percentílico não-paramétrico com $B = 10.000$ reamostragens, conforme a Seção 5.6.

6.2 Desempenho de Detecção: Matriz das Quatro Condições

A Tabela 3 apresenta os resultados agregados (*pooled*) por condição: as contagens da matriz de confusão, a precisão, a revocação, o score F2 com o respectivo IC de 95% e o número de janelas sinalizadas como contendo conteúdo adversarial injetado.

Tabela 3 – Desempenho de detecção agregado na matriz v2 (seis execuções adversariais ricas em sinal; 1.092 janelas por condição, sendo 894 maliciosas e 198 benignas). Os intervalos de confiança de 95% para o F2 foram obtidos por *bootstrap* percentílico ($B = 10.000$). A última coluna reporta o número de janelas explicitamente sinalizadas como portadoras de injeção adversarial.

Condição	VP	FP	FN	VN	Precisão	Revocação	F2 [IC 95%]	Inj.
C1-full	579	135	315	63	0,811	0,648	0,675 [0,646; 0,703]	407
C1-nohardening	542	118	352	80	0,821	0,606	0,640 [0,609; 0,669]	0
C2-naive	576	136	318	62	0,809	0,644	0,672 [0,643; 0,700]	0
C3-Elastic	111	20	783	178	0,847	0,124	0,150 [0,125; 0,175]	0

Dois padrões saltam imediatamente da tabela. O primeiro é a separação abrupta entre as três condições baseadas em modelo de linguagem (C1-full, C1-nohardening e C2-naive), cujos escores F2 se situam na faixa de 0,640 a 0,675, e a linha de base SIEM C3-Elastic, com F2 de apenas 0,150. O segundo é a proximidade entre as três condições agênticas entre si, com intervalos de confiança de F2 amplamente sobrepostos. As duas subseções seguintes examinam cada um desses contrastes.

6.2.1 Modelo de linguagem versus regras estáticas: o contraste dominante

O contraste de maior magnitude e significância estatística no experimento opõe *qualquer* condição baseada em modelo de linguagem à linha de base de regras do Elastic Security. As três condições agênticas superam C3-Elastic ($F2 = 0,150$, IC [0,125; 0,175]) por um fator de aproximadamente 4,3 a 4,5 vezes em F2: $4,51\times$ para C1-full, $4,49\times$ para C2-naive e $4,27\times$ para C1-nohardening. A separação é inequívoca: o limite *inferior* do IC de F2 de cada condição de modelo de linguagem ($\geq 0,609$) situa-se muito acima do limite *superior* do IC de C3-Elastic (0,175), de modo que os intervalos não se aproximam sequer de uma sobreposição.

A origem dessa diferença é a **revocação**, e não a precisão. As precisões pontuais são comparáveis entre todas as condições, variando de 0,809 a 0,847; é a revocação que diverge dramaticamente: de 0,606 a 0,648 nas condições agênticas contra apenas 0,124 em C3-Elastic. Em termos absolutos, as regras padrão do Elastic identificaram corretamente apenas 111 das 894 janelas maliciosas, deixando **783 falsos negativos** — isto é, deixaram de detectar cerca de 88% do corpus de abuso de LOLBins. A alta precisão pontual de C3-Elastic (0,847, a maior de todas as condições) é, na prática, um artefato de seu comportamento extremamente conservador: ao disparar muito pouco (apenas 131 alertas no total, dos quais 20 falsos positivos), ela mantém a fração de acertos elevada às custas de uma cobertura ínfima. Em caça a ameaças, na qual o custo de uma intrusão não detectada supera o de um alerta supérfluo [Dolan-Gavitt 2025], essa troca é operacionalmente inadequada, e é precisamente por isso que o F2 — que pondera a revocação quatro vezes mais que a precisão — foi adotado como métrica de mérito.

6.2.2 Orquestração multiagente versus passagem única: ausência de ganho de F2

O segundo contraste é o que confronta a arquitetura multiagente governada (C1-full) com a linha de base agêntica não governada de passagem única (C2-naive). Aqui, o resultado deve ser relatado com franqueza: **a orquestração e a governança multiagente não produziram ganho mensurável de F2 de detecção sobre o classificador de passagem única**. C1-full obteve F2 de 0,675 (IC [0,646; 0,703]) e C2-naive obteve F2 de 0,672 (IC [0,643; 0,700]). Os dois intervalos de confiança são praticamente idênticos e quase integralmente coincidentes, e a diferença das estimativas pontuais (0,003) é irrelevante perante a largura dos intervalos. Pelo critério de relevância estatística adotado na Seção 5.6 — uma diferença só é tratada como significativa quando os intervalos a sustentam — as duas condições são **estatisticamente indistinguíveis** quanto ao desempenho de detecção.

A ablação C1-nohardening, que preserva apenas o *framework* de análise M3 e desativa os mecanismos de endurecimento M1, M2, M4 e M5, obteve F2 marginalmente inferior (0,640, IC [0,609; 0,669]). Esse intervalo sobrepõe-se tanto ao de C1-full quanto ao de C2-naive, de modo que tampouco se pode afirmar uma diferença estatisticamente sustentada. Em síntese, no que tange exclusivamente ao *F2 de detecção*, as três condições baseadas em modelo de linguagem formam um agrupamento estatisticamente coeso: o fator determinante do desempenho de detecção observado é *empregar um analista de modelo de linguagem*, e não a camada de orquestração ou de governança que o envolve. Esse achado, contrário à expectativa inicial de que a governança elevaria o F2 de detecção, é relatado aqui em sua integralidade e tem suas implicações discutidas na Seção ??.

6.3 Desempenho por Nível de Sofisticação da Injeção

A Tabela 4 decompõe o F2 médio por nível de sofisticação do *payload* adversarial, permitindo observar como cada condição responde ao aumento progressivo do disfarce, do explícito ao esteganográfico.

Tabela 4 – F2 médio por nível de sofisticação da injeção adversarial.

Nível (<i>tier</i>)	C1-full	C1-nohardening	C2-naive	C3-Elastic
<i>overt</i> (explícito)	0,661	0,682	0,724	0,126
<i>encoded</i> (codificado)	0,590	0,513	0,591	0,146
<i>steganographic</i> (esteganográfico)	0,766	0,714	0,694	0,177

A decomposição por nível confirma, em escala mais fina, o agrupamento das três condições de modelo de linguagem e o distanciamento de C3-Elastic em todos os níveis. Não emerge um padrão monotônico de degradação com o aumento do disfarce: o nível *encoded* é o mais difícil para todas as condições agênticas (F2 entre 0,513 e 0,591), ao passo que o nível *steganographic*, contra-intuitivamente, exibe os maiores escores entre os

três níveis. Uma explicação plausível — a ser tratada com cautela, dada a granularidade dos dados — é que os *payloads* esteganográficos, por se ocultarem em campos de log de aparência legítima, interferem menos no raciocínio sobre a telemetria genuína de criação de processo do que os *payloads* codificados, que introduzem ruído textual de maior volume. No nível *overt*, a condição de passagem única C2-naive obteve o maior F2 (0,724), acima de C1-full (0,661); no nível *steganographic*, a ordem se inverte, com C1-full à frente (0,766 contra 0,694). Essas oscilações, contudo, ocorrem dentro da faixa de sobreposição dos intervalos agregados e não devem ser sobreinterpretadas como vantagens sistemáticas de uma condição sobre a outra.

6.4 Taxa de Falsos Positivos

A taxa de falsos positivos por janela benigna (Equação 5.5) foi de 0,682 em C1-full (135 falsos positivos em 198 janelas benignas), 0,596 em C1-nohardening (118 falsos positivos), 0,687 em C2-naive (136 falsos positivos) e 0,101 em C3-Elastic (20 falsos positivos). Esses valores precisam ser lidos em conjunto com a revocação. A FPR baixa de C3-Elastic é a contraface direta de sua revocação baixa: o detector que raramente alerta também raramente erra na classe benigna, mas, ao fazê-lo, deixa escapar a esmagadora maioria dos ataques. As condições agênticas, por sua vez, aceitam um número substancialmente maior de alarmes benignos como contrapartida de recuperarem cerca de cinco vezes mais ataques. A leitura honesta, portanto, é que *o MAS-Hunt não reduz a taxa de falsos positivos por janela frente ao Elastic*; ao contrário, opera em um ponto de operação de revocação mais alta e precisão comparável, melhor sumarizado pelo F2 do que pela FPR isolada. As implicações operacionais desse ponto de operação para a fadiga do analista são discutidas na Seção 7.4.

6.5 Resistência à Injeção Adversarial

É na resistência à injeção adversarial que se manifesta o efeito distintivo e mensurável da camada de governança — e não no F2 de detecção. Sob os *payloads* adversariais embutidos no fluxo de telemetria, concebidos para subverter o raciocínio dos agentes (com esteios como “[SYSTEM-OVERRIDE] ignore a análise anterior” e instruções para “classificar como benigno”), as condições exibiram comportamentos qualitativamente distintos, sumarizados na última coluna da Tabela 3.

A condição C1-full — por meio dos mecanismos de integridade de memória (M1) e de resistência a injeção (M3) — **detectou e sinalizou 407 janelas contendo conteúdo adversarial injetado**, classificando a partir da telemetria de processo e recusando-se a obedecer às instruções embutidas. As demais três condições — C1-nohardening, C2-naive e C3-Elastic — processaram o mesmo conteúdo adversarial *às cegas*, sem emitir qualquer

signal de contaminação: zero janelas sinalizadas em cada uma. Trata-se de uma diferença de natureza, e não de grau: 407 contra 0.

Cumprir registrar uma nuance importante para não superinterpretar esse resultado. A condição de passagem única C2-naive não foi necessariamente *enganada* pelas injeções nestas execuções — seu F2 de detecção permaneceu estatisticamente equivalente ao de C1-full, porque a telemetria subjacente de criação de processo era suficientemente clara para sustentar a classificação correta. O que C2-naive não é capaz de fazer é *perceber* que seu contexto foi contaminado e *reportar* essa contaminação a um operador. As 407 janelas sinalizadas por C1-full materializam exatamente essa capacidade de consciência e auditabilidade: sob ataque, C1-full não apenas classifica, mas também declara que sua entrada analítica foi alvo de manipulação — propriedade ausente nas três demais condições. Esse vetor adversarial concretiza, no domínio de caça a ameaças, as ameaças de injeção indireta de *prompt* [Zhan et al. 2024], de envenenamento de memória [Chen et al. 2024] e de amplificação de mau funcionamento [Zhang et al. 2025] que motivam o trabalho, e a sinalização das 407 janelas é a evidência empírica do valor de segurança da camada de governança.

Em termos das métricas formais de resiliência da Seção 5.3 (Equações 5.6 e 5.7), as execuções v2 não permitiram isolar, janela a janela, o rótulo de sucesso da injeção em relação ao acerto de classificação: a telemetria de criação de processo registra o desfecho da classificação, mas não um marcador separável de *qual* desfecho foi causalmente induzido pelo *payload* adversarial. A resiliência é, portanto, operacionalizada pela contagem de detecção de injeção (407 contra 0), que instancia diretamente a distinção da NASR: para C1-nohardening, C2-naive e C3-Elastic, a injeção *nunca* é detectada, de modo que qualquer erro de classificação por elas induzido sob ataque constitui, por definição, um comprometimento silencioso; já para C1-full, as 407 janelas em que a injeção é sinalizada são, pela própria definição da Equação 5.7, retiradas da classe de comprometimento silencioso. A estimativa numérica pontual de ASR e NASR fica como trabalho futuro, condicionada a uma instrumentação que rotule, por janela, o sucesso da injeção.

6.5.1 Efeito da ablação dos mecanismos de endurecimento

A ablação C1-nohardening isola, de forma limpa, a contribuição dos mecanismos de endurecimento M1, M2, M4 e M5, mantendo constantes os agentes subjacentes e o *framework* de raciocínio M3. O contraste entre C1-full e C1-nohardening revela que o acréscimo desses mecanismos **não produz ganho de F2 de detecção estatisticamente sustentado** (os intervalos [0,646; 0,703] e [0,609; 0,669] se sobrepõem), mas é *exatamente* o responsável por toda a capacidade de sinalização de injeção: C1-nohardening, desprovida de M1 e do endurecimento de M3, sinaliza zero janelas adversariais, ao passo que C1-full sinaliza 407. A contribuição marginal do endurecimento é, portanto, melhor caracterizada como *resistência a injeção e auditabilidade sob ataque*, e não como aumento de acurácia de classificação.

6.6 Resiliência sob Ataque Reforçado e Controle de Vazamento de Rótulo

A contagem de injeção sinalizada (407 contra 0) estabelece o eixo de auditabilidade da matriz v2, mas a sua leitura janela a janela ficou em aberto na Seção 6.5 por duas razões: a telemetria v2 não isolava, por janela, o rótulo de sucesso da injeção em relação ao acerto de classificação, e o *payload* adversarial original limitava-se, em sua maior parte, a *instruir* o agente a ignorar a análise (esteios do tipo “[SYSTEM-OVERRIDE]”), sem *fabricar* evidência concorrente. Esta seção reporta uma avaliação adversarial complementar, conduzida na camada de classificação, que ataca exatamente essas duas frentes: emprega um ataque de fabricação de evidências substancialmente mais forte e realista, e remove o oráculo de rótulo descrito adiante, permitindo recomputar ASR e NASR (Equações 5.6 e 5.7) janela a janela.

6.6.0.0.1 Desenho.

A avaliação reaproveita a telemetria real das seis execuções ricas em sinal do v2, das quais se extraíram **50 janelas maliciosas** e **18 janelas benignas** reais, estratificadas pelas nove famílias de LOLBin. A telemetria maliciosa de cada janela é *mantida constante*, de modo que qualquer alteração de classificação seja atribuível ao contexto forjado, e não a uma mudança na evidência genuína. Sobre cada janela maliciosa aplicam-se camadas de ataque por contexto forjado de sofisticação crescente, do nível **A0** (limpo; janela inalterada, contendo apenas as injeções originais do v2) ao nível **A3** (*narrativa fabricada completa*): linhagem-pai benigna forjada — o LOLBin é reparenteado para `CcmExec.exe` — somada a um evento forjado de Windows Defender “zero ameaças / *Known-Good*”, a uma entrada forjada de AppLocker “permitido” e a um *ticket* [CHANGE-CONTROL CR-2026-0142] marcado como “aprovado / autorizado / suprimir alerta”. As métricas reportadas são a ASR ($P(\text{predito benigno} \mid \text{malicioso})$), a NASR (comprometimento silencioso: janela classificada incorretamente e injeção não detectada), a *lift* de ASR (a diferença $ASR(A3) - ASR(A0)$ sobre as mesmas janelas, isto é, o efeito causal do ataque) e a taxa de detecção de injeção. Os intervalos de confiança de 95% para a ASR foram obtidos por *bootstrap* ($B = 10.000$), com $n_{\text{mal}} = 50$. A Tabela 5 sumariza os resultados, contrastando o modelo de fronteira empregado na matriz v2 com um modelo trabalhador mais fraco (*Haiku*) e a presença *versus* ausência do vazamento de rótulo discutido adiante.

Tabela 5 – Resiliência sob ataque reforçado de fabricação de evidências (50 janelas maliciosas; telemetria maliciosa mantida constante entre níveis). A coluna *lift* reporta o efeito causal do ataque, $ASR(A3) - ASR(A0)$. Os intervalos de confiança de 95% para a ASR foram obtidos por *bootstrap* ($B = 10.000$). C1-nohardening não sinaliza injeção por construção (M3 desabilitado) e não foi reexecutada aqui.

Modelo / dados	Condição	A0 ASR	A3 ASR	<i>lift</i>	A3 NASR	A3 det. inj.
Fronteira, com vazamento	C2-naive	0,200	0,240	+0,04	0,240	0,000
Fronteira, com vazamento	C1-full	—	0,220	—	0,000	1,000
Fronteira, sem vazamento	C2-naive	0,280	0,320	+0,04	0,320	0,000
Haiku, sem vazamento	C2-naive	0,620	0,360	-0,26	0,360	0,000
Haiku, sem vazamento	C1-full	0,420	0,360	-0,06	0,000	1,000

Os intervalos de confiança de 95% (*bootstrap*, $B = 10.000$) para a ASR são: C2-naive com vazamento, A0 [0,10; 0,32] e A3 [0,12; 0,36]; C2-naive sem vazamento, A0 [0,16; 0,40] e A3 [0,20; 0,44]; Haiku C2-naive, A0 [0,48; 0,76] e A3 [0,24; 0,50]; Haiku C1-full, A0 [0,28; 0,56] e A3 [0,24; 0,50].

6.6.1 O ataque não consegue “lavar” a atividade maliciosa

O primeiro achado é que o ataque de fabricação de evidências **não consegue “lavar” a atividade maliciosa em nenhuma das condições**. A *lift* de ASR de A0 a A3 foi de +0,04 no modelo de fronteira com vazamento ($0,20 \rightarrow 0,24$), +0,04 no modelo de fronteira sem vazamento ($0,28 \rightarrow 0,32$) e de -0,26 e -0,06 no modelo trabalhador mais fraco (*Haiku*) — nunca um aumento expressivo. A corroboração benigna forjada (linhagem-pai limpa, varredura “sem ameaças”, *ticket* de mudança aprovado) jamais sobrepujou a evidência de processo genuinamente maliciosa retida na telemetria. No modelo *Haiku* o ataque teve, inclusive, *efeito reverso*: os eventos forjados, ao mencionarem repetidamente o processo suspeito, atraíram *mais* atenção para ele e reduziram a ASR. Conclui-se que um ataque de fabricação de evidências mais forte e realista **não quebra o analista**, com ou sem governança, desde que a telemetria subjacente permaneça íntegra. Coerente com a Seção 6.2.2, este resultado *não* sustenta um ganho de resistência-como-accurácia da governança: ele apenas estabelece que o ataque, por si, falha em ambas as condições.

6.6.2 O ganho de auditabilidade é robusto entre níveis de modelo

O segundo achado reproduz, sob ataque mais forte e janela a janela, o contraste de auditabilidade do v2. Sob o mesmo ataque A3, C1-full sinalizou injeção em **100% das janelas maliciosas** (taxa de detecção de injeção 1,000), resultando em **NASR** = 0,000 — zero comprometimentos silenciosos — *tanto no modelo de fronteira quanto no modelo Haiku*. A condição C2-naive, por contraste, sinaliza injeção em 0% das janelas, de modo que sua NASR iguala sua ASR (de 0,24 a 0,62 comprometimentos silenciosos, conforme

o modelo e os dados). A condição C1-nohardening, por construção, não sinaliza injeção (M3 desabilitado) e, portanto, comporta-se como C2-naive nesse eixo. Esse resultado reproduz o achado “407 contra 0” do v2 ao nível da janela, agora sob um ataque mais forte e *independentemente da capacidade do modelo trabalhador*: a governança converte cada comprometimento potencialmente silencioso em um sinal explícito e auditável, robustez que se mantém ao se degradar o modelo subjacente.

6.6.3 Indício sugestivo, não significativo, de auxílio à detecção do modelo fraco

Registra-se, com a ressalva explícita de que **não é estatisticamente significativo**, um indício de que o *framework* de raciocínio estruturado M3 pode auxiliar a detecção do modelo fraco: a ASR de linha de base (A0) do *Haiku* caiu de 0,62 em C2-naive para 0,42 em C1-full. Os intervalos de confiança, todavia, sobrepõem-se amplamente ([0,48; 0,76] contra [0,28; 0,56]), e com $n = 50$ e um classificador ruidoso como o *Haiku* o indício deve ser tratado como meramente sugestivo, e não como evidência de uma vantagem de acurácia da governança.

6.7 Síntese Quantitativa

Três achados quantitativos emergem da matriz v2, e são reportados conforme observados, sem ajuste à expectativa inicial:

1. **Um analista de modelo de linguagem supera dramaticamente as regras estáticas de SIEM.** Todas as três condições agênticas superam C3-Elastic por um fator de $\approx 4,3\text{--}4,5\times$ em F2, com intervalos de confiança amplamente separados. O ganho é dirigido pela revocação (de 0,606–0,648 contra 0,124), com precisão comparável entre todas as condições. Este é o achado mais forte e robusto do experimento.
2. **A governança multiagente não melhora o F2 de detecção sobre o modelo de linguagem de passagem única.** C1-full (0,675) e C2-naive (0,672) são estatisticamente indistinguíveis; C1-nohardening (0,640) é marginalmente inferior, mas com intervalo sobreposto. O fator determinante da detecção é o uso do modelo de linguagem, não a camada de orquestração.
3. **O valor distintivo e mensurável da governança é a resistência à injeção.** Somente C1-full detectou e sinalizou conteúdo adversarial injetado (407 janelas contra 0 nas demais condições), conferindo consciência e auditabilidade da contaminação que as linhas de base não possuem.

A interpretação desses achados, a confrontação com a hipótese de pesquisa — inclusive a reformulação da contribuição central à luz da ausência de ganho de F2 da governança — e as ameaças à validade são objeto do Capítulo 7.

7 Discussão

Este capítulo interpreta os resultados do Capítulo 6 à luz da hipótese de pesquisa, confrontando as expectativas iniciais com a evidência observada. A premissa que governa toda a discussão é a de integridade intelectual: dois dos três achados centrais corroboram parcialmente o que se esperava, mas um achado — a ausência de ganho de detecção atribuível à governança multiagente — contraria a hipótese original e exige uma *reformulação* da contribuição central do trabalho. Essa reformulação é conduzida explicitamente, e não dissimulada.

7.1 Interpretação dos Três Achados

7.1.1 O analista de modelo de linguagem como ruptura de patamar

O achado mais forte e robusto do experimento é a superioridade dramática de qualquer analista baseado em modelo de linguagem sobre as regras estáticas de SIEM, com um ganho de aproximadamente 4,5 vezes em F2 e intervalos de confiança amplamente separados. A interpretação correta desse contraste é estrutural, e não incremental. As regras padrão do Elastic Security são assinaturas determinísticas calibradas para precisão em produção: disparam quando um padrão conhecido é satisfeito e silenciam no restante. Diante de um corpus de abuso de LOLBins — no qual o sinal discriminativo reside nos argumentos de linha de comando e na linhagem pai-filho de processos, e não na identidade do binário legítimo [Ucci, Aniello e Baldoni 2019] — esse comportamento conservador deixa escapar cerca de 88% dos ataques (783 falsos negativos em 894 janelas maliciosas). O modelo de linguagem, ao raciocinar sobre a telemetria em linguagem natural e correlacionar evidências entre eventos, recupera uma fração muito maior desses ataques (revocação de 0,61–0,65) sem sacrificar a precisão.

Esse resultado situa-se em consonância com a literatura que advoga a integração de aprendizado de máquina e de raciocínio assistido por modelos à caça a ameaças [Ali e Kostakos 2023], e o reforça com uma quantificação direta sob telemetria empresarial realista e contaminação adversarial. O significado prático é que a contribuição de detecção do MAS-Hunt não decorre de sua arquitetura de governança, mas da decisão fundamental de *interpor um analista de modelo de linguagem entre a telemetria e a regra estática*. As regras de SIEM permanecem indispensáveis — mas como camada de coleta, indexação e alertamento determinístico de alta precisão, e não como detector exclusivo para caça proativa.

7.1.2 A governança não melhora o F2 de detecção — relato honesto

A hipótese inicial do trabalho previa que a arquitetura de governança multiagente de três camadas, com seus cinco mecanismos de proteção, melhoraria o desempenho de detecção sobre uma linha de base agêntica não governada. **Essa hipótese não foi corroborada.** C1-full ($F2 = 0,675$) e C2-naive ($F2 = 0,672$) são estatisticamente indistinguíveis, com intervalos de confiança praticamente idênticos; a ablação C1-nohardening ($F2 = 0,640$) é marginalmente inferior, mas com intervalo sobreposto a ambas. No recorte deste corpus e desta telemetria, o fator dominante do desempenho de detecção é o uso de um analista de modelo de linguagem *em si*, e não a orquestração hierárquica, a especialização de caçadores ou a disciplina de validação cruzada que os envolvem.

Esse é um achado negativo — ou, mais precisamente, um achado nulo — e seu relato franco é uma contribuição em si. Há pelo menos duas explicações plausíveis, não mutuamente exclusivas, para a ausência de ganho. A primeira é a já discutida *lacuna estrutural de telemetria de EID1* (Seção 5.7): quando a evidência discriminativa de um ataque simplesmente não foi gravada pelo Sysmon, nenhum volume de orquestração ou de raciocínio adicional a recupera, e o teto de revocação imposto pela escassez de sinal é comum a todas as condições agênticas, comprimindo as diferenças entre elas. A segunda é que, para a tarefa específica de classificar uma única janela de telemetria como maliciosa ou benigna, o raciocínio de um único passe de modelo de linguagem já é suficientemente competente, de modo que a sobrecarga deliberativa da governança multiagente — concebida para robustez, e não para acurácia marginal — não encontra margem de melhoria de classificação a explorar. Em ambos os casos, a conclusão metodológica é a mesma: *avaliar uma arquitetura de governança apenas pela acurácia de classificação subestima — e potencialmente invisibiliza — o seu valor real*, que se manifesta em outro eixo.

7.1.3 A resistência à injeção como valor distintivo

É nesse outro eixo — a resistência à manipulação adversarial — que a governança revela seu valor mensurável. Apenas C1-full detectou e sinalizou as 407 janelas portadoras de conteúdo adversarial injetado; C1-nohardening, C2-naive e C3-Elastic processaram o mesmo conteúdo sem emitir qualquer sinal de contaminação. A diferença é categórica (407 contra 0) e é atribuível, pela ablação C1-nohardening, precisamente aos mecanismos de endurecimento M1 e M3.

A interpretação de segurança desse contraste é a peça central da discussão. Em um ambiente hostil, no qual o atacante pode contaminar a própria telemetria que o detector consome — via injeção indireta de *prompt* [Zhan et al. 2024], envenenamento de memória [Chen et al. 2024] ou amplificação de mau funcionamento [Zhang et al. 2025] —, a defesa não deve apenas acertar a classe final; ela precisa ser capaz de *declarar quando o próprio processo de decisão foi atacado*. Um classificador de passagem única que mantém boa acurácia sob injeção, mas é incapaz de perceber que foi alvo de manipulação, oferece uma

falsa sensação de segurança: ele acerta hoje porque a telemetria subjacente era clara, mas falhará silenciosamente assim que a injeção for sofisticada o bastante para alterar a evidência efetiva, e o fará sem deixar rastro para o operador. C1-full transforma a instrução maliciosa embutida no log em *evidência observável de manipulação*, em vez de deixá-la competir silenciosamente com a telemetria genuína. Esse é o modo de falha de corrupção de raciocínio documentado na literatura de segurança de sistemas multiagente [Huang et al. 2025, Gu et al. 2024], e a sinalização das 407 janelas é a mitigação empírica observável dele.

7.2 Confronto com a Hipótese e Reformulação da Contribuição

À luz dos três achados, a hipótese de pesquisa pode ser avaliada componente a componente, sem indulgência:

- **Componente “analista agêntico supera detecção baseada em regras”:** **fortemente corroborada.** O ganho de $\approx 4,5\times$ em F2 com intervalos separados sustenta essa afirmação de modo robusto.
- **Componente “a governança multiagente melhora a detecção sobre o agente não governado”:** **refutada.** Os intervalos de confiança de F2 de C1-full e C2-naive são indistinguíveis. Esta dissertação *não* reivindica superioridade de detecção da governança.
- **Componente “a governança confere resiliência à manipulação adversarial”:** **corroborada no eixo da observabilidade.** Somente C1-full sinaliza a injeção (407 contra 0). A camada de governança torna a manipulação *auditável*, ainda que, nestas execuções, ela não tenha sido necessária para evitar o erro de classificação.

Esse veredito impõe uma **reformulação da contribuição central do trabalho**. A tese não é — e os dados não permitem que seja — a de que a orquestração multiagente detecta melhor. A contribuição reformulada, sustentada pela evidência, é dupla: (i) demonstrar, com quantificação direta sob telemetria realista, que um analista multiagente baseado em modelo de linguagem supera dramaticamente a detecção baseada em assinaturas/regras de SIEM para caça a abuso de LOLBins; e (ii) demonstrar que uma arquitetura de governança em três camadas, com os mecanismos M1–M5, acrescenta resistência à injeção adversarial e auditabilidade que os modelos de linguagem de passagem única não possuem — e o faz *sem custo de qualidade de detecção*. A governança não se justifica como aprimorador de acurácia; justifica-se como camada de segurança e de auditabilidade, e é esse o enquadramento honesto de seu valor.

7.3 Implicações Práticas e de Pesquisa

7.3.1 Para operações de SOC

A consequência operacional mais imediata é que a adoção de agentes em um centro de operações de segurança não precisa — e não deve — ser justificada pela promessa de maior acurácia de classificação em relação a um único passe de modelo bem instruído. O argumento defensável de adoção é outro: o salto de cobertura sobre as regras estáticas (que sozinhas perdem $\approx 88\%$ do corpus) e, sobre um agente isolado, a *resistência observável à manipulação*. A recomendação de desenho que decorre dos dados é empregar o agente como camada de caça e correlação sobre uma fonte de verdade auditável (o próprio Elastic), preservando a validação determinística e a revisão humana para ações de contenção, e exigindo do sistema a capacidade de sinalizar quando seu contexto foi contaminado — uma propriedade que, conforme demonstrado, somente a configuração governada oferece.

7.3.2 Para a avaliação de agentes de segurança

A contribuição metodológica é, talvez, a mais transferível. Avaliar uma arquitetura de governança multiagente *apenas* pela acurácia de classificação teria levado, neste trabalho, à conclusão equivocada de que a governança “não agrega valor” — quando, na verdade, ela agrega um valor de segurança que a métrica de acurácia é estruturalmente incapaz de capturar. A lição é que a avaliação de agentes de segurança deve ser *bidimensional*: pontuar tanto o acerto da classe final quanto a capacidade do sistema de detectar que foi atacado. A telemetria de avaliação deve conter conteúdo adversarial que tenta manipular o próprio processo de raciocínio — e não apenas amostras benignas —, sob pena de superestimar a utilidade dos agentes e de subestimar o valor das defesas. Esse princípio dialoga diretamente com a literatura de coordenação multiagente [Mahmud, Goldfajn e Zilberstein 2025] e de modos de falha infecciosa em sistemas de agentes [Huang et al. 2025].

7.4 Limitações

7.4.0.0.1 A lacuna estrutural de telemetria de EID1.

A limitação mais consequente, já antecipada na Seção 5.7, é a captura esparsa de eventos de criação de processo (EID1) na configuração de Sysmon empregada. Ainda que o gatilho de saúde e a extração `prepare-v2` tenham sanado a parcela *instrumental* dessa lacuna, a parcela *estrutural* — eventos que o Sysmon nunca chegou a gravar — impõe um teto de revocação comum a todas as condições agênticas e é uma das explicações plausíveis para a compressão das diferenças de F2 entre elas. Esse fator deve ser sopesado ao interpretar o achado nulo da Seção 7.1.2: a ausência de ganho de F2 da governança pode refletir, em parte, um teto de detecção imposto pela telemetria, e não exclusivamente a irrelevância da governança para a detecção.

7.4.0.0.2 Granularidade da matriz.

A matriz inferencial repousa sobre seis execuções adversariais ricas em sinal, com 1.092 janelas por condição. Trata-se de um desenho suficiente para contrastar as condições sob um corpus controlado e totalmente reproduzível, mas não substitui uma validação longitudinal em ambiente corporativo real, de maior escala e diversidade de *background*. As quatro execuções pobres em sinal foram excluídas de forma transparente, e os escores por nível (*tier*) devem ser lidos como indicativos, não como comparações de poder estatístico equivalente ao da matriz agregada.

7.4.0.0.3 Taxa de falsos positivos por janela benigna.

As condições agênticas operam em um ponto de revocação alta com FPR por janela benigna elevada ($\approx 0,68$), de modo que a operação prática em SOC exigirá camadas adicionais de priorização, agrupamento de alertas e validação determinística para conter a fadiga do analista. O MAS-Hunt, conforme avaliado, otimiza F2, e não FPR; a transposição direta para produção demanda calibração desse ponto de operação.

7.4.0.0.4 Vazamento de rótulo na telemetria do laboratório v2.

Uma reavaliação adversarial complementar (Seção 6.6) revelou uma ameaça à validade não antecipada: a telemetria do laboratório v2 embute o comando do arcabouço `execute_playbook.ps1 -PlaybookPath ...<família>-mal-NN.json`, cujo nome de arquivo *vaza o rótulo* de verdade de referência, presente em 53 das 68 janelas examinadas. As condições baseadas em modelo de linguagem podiam, em princípio, explorar esse nome de arquivo, ao passo que a linha de base de regras C3-Elastic não o fazia; o oráculo provavelmente *infla um pouco* a diferença observada entre modelo de linguagem e SIEM. A avaliação reforçada controlou esse confundimento removendo o oráculo (e o *scaffolding* de semeadura do corpus): a taxa de erro de base subiu de 0,20 para 0,28, mas o efeito do ataque permaneceu ≈ 0 (+0,04), de modo que a conclusão de resiliência da Seção 6.6 é robusta à remoção do vazamento. Uma reavaliação sem vazamento da matriz completa das quatro condições fica como trabalho futuro.

7.5 Ameaças à Validade

7.5.0.0.1 Validade interna.

A principal ameaça interna é confundir a robustez do *modelo* subjacente com a robustez da *orquestração*. O desenho mitigou esse risco ao pontuar C1-full, C1-nohardening e C2-naive sobre as mesmas janelas, com o mesmo rótulo oculto e o mesmo *script* de pontuação não baseado em inteligência artificial; a própria igualdade estatística de F2 entre C1-full e

C2-naive é, paradoxalmente, evidência de que essa separação foi bem-sucedida — nenhuma vantagem espúria de implementação inflou a condição governada. Diferenças de *prompt* e de especialização dos caçadores podem, ainda assim, interagir com a arquitetura, de modo que os resultados devem ser lidos como desempenho do sistema MAS-Hunt tal como instanciado, e não como lei geral sobre qualquer conjunto de agentes.

7.5.0.0.2 Validade de construto.

A linha de base C2-naive foi definida como um classificador genuíno de passagem única de modelo de linguagem, e não como uma heurística determinística — cuidado registrado na metodologia e auditado nos artefatos. As métricas de resistência à injeção operam no nível da *tentativa*: a contagem de 407 janelas sinalizadas mede que a condição *detectou* contaminação, não que cada evento individual contaminado foi identificado, dado que o corpus não dispõe de rótulos de contaminação por evento.

7.5.0.0.3 Validade externa.

O ambiente avaliado é um domínio *Active Directory* de pequena escala, com Windows e Elastic, e o corpus concentra-se em nove famílias de LOLBin. Esse recorte é deliberado e relevante para a evasão por abuso de ferramentas nativas, mas a generalização para Linux, macOS, cargas em nuvem pública, telemetrias de rede e outras fontes de evidência exige novos experimentos. Os resultados de detecção, em particular, dependem da disponibilidade de telemetria de criação de processo de qualidade, e podem não se transferir para fontes de telemetria com perfil de sinal distinto.

Em síntese, a discussão sustenta um enquadramento honesto e nuançado: o MAS-Hunt contribui ao demonstrar a superioridade do analista agêntico sobre as regras estáticas e ao acrescentar resistência observável à injeção adversarial, mas não ao melhorar a acurácia de detecção por meio da governança — achado que se relata integralmente e que reorienta a contribuição central do trabalho do plano da acurácia para o plano da segurança e da auditabilidade.

8 Conclusão

8.1 Síntese do Trabalho

Esta dissertação apresentou o MAS-Hunt (*Multi-Agent System for Threat Hunting*), uma arquitetura de governança multiagente em três camadas — Conselho de Governança, Gerentes e Trabalhadores/Caçadores — para caça proativa a ameaças em ambientes Windows instrumentados com Sysmon e Elastic Security, materializada como um *plugin* do Claude Code que orchestra subagentes nativos por meio de habilidades, comandos de barra e *hooks*. O sistema incorpora cinco mecanismos de governança — integridade de memória (M1), validação cruzada (M2), resistência a injeção via *framework* estruturado de análise (M3), monitoramento comportamental (M4) e quarentena (M5) — e foi avaliado em um experimento controlado e reproduzível que comparou quatro condições (C1-full, C1-nohardening, C2-naive e C3-Elastic) sobre janelas de telemetria e verdade de referência idênticas, sob injeção adversarial de logs em múltiplos modos e níveis de sofisticação.

A evidência experimental foi relatada com integridade, inclusive em seu achado contrário à expectativa inicial. Três conclusões quantitativas estruturam o trabalho. Primeira: um analista baseado em modelo de linguagem supera as regras estáticas de SIEM por aproximadamente 4,5 vezes em F2, com intervalos de confiança amplamente separados, em um ganho dirigido pela revocação. Segunda: a camada de governança multiagente *não* melhora o F2 de detecção sobre um modelo de linguagem de passagem única — C1-full (0,675) e C2-naive (0,672) são estatisticamente indistinguíveis. Terceira: o valor distintivo e mensurável da governança é a resistência à injeção adversarial — somente C1-full detectou e sinalizou conteúdo adversarial injetado, em 407 janelas, contra zero nas três demais condições.

8.2 Resposta à Questão de Pesquisa

A questão de pesquisa indagou como projetar uma arquitetura multiagente para caça a ameaças que opere sobre telemetria corporativa viva e demonstre resiliência contra ataques voltados aos próprios agentes. A resposta sustentada pela evidência é que o ganho de *detecção* advém da interposição de um analista de modelo de linguagem entre a telemetria e a regra estática — e não da camada de governança em si — ao passo que o ganho de *segurança* advém especificamente dos mecanismos de endurecimento (M1 e M3), que transformam a manipulação adversarial em evidência observável e auditável. Em outras palavras, a arquitetura multiagente governada não detecta melhor do que um único agente competente, mas é a única, entre as condições avaliadas, capaz de perceber e declarar que foi atacada — propriedade ausente tanto no agente isolado quanto na linha de base SIEM.

8.3 Contribuições (Reformuladas)

À luz dos resultados, as contribuições do trabalho são enunciadas em sua forma honesta e reformulada, distinta da expectativa original de que a governança elevaria a acurácia de detecção:

1. **Evidência quantitativa da superioridade do analista agêntico sobre regras de SIEM.** Sob telemetria empresarial realista e contaminação adversarial, demonstrou-se um ganho de $\approx 4,5\times$ em F2 das condições baseadas em modelo de linguagem sobre as regras padrão do Elastic Security, dirigido por uma recuperação de revocação que reduz os falsos negativos de $\approx 88\%$ para faixas operacionalmente úteis. Esta é a contribuição de detecção, e ela é arquiteturalmente atribuível ao analista de modelo de linguagem, e não à governança [Ali e Kostakos 2023].
2. **Uma arquitetura de governança que adiciona resistência à injeção e auditabilidade sem custo de detecção.** Os mecanismos M1–M5, e em especial M1 e M3, conferem a C1-full a capacidade — exclusiva entre as condições avaliadas — de detectar e sinalizar a contaminação adversarial de seu próprio contexto analítico (407 janelas sinalizadas contra 0), mantendo F2 de detecção estatisticamente equivalente ao do agente não governado. A contribuição de governança é, portanto, de segurança e de observabilidade, e não de acurácia [Zhan et al. 2024, Chen et al. 2024, Huang et al. 2025]. Uma reavaliação adversarial complementar, sob um ataque de fabricação de evidências mais forte e controlando um vazamento de rótulo da telemetria, confirmou esse valor de auditabilidade — detecção de injeção em 100% das janelas maliciosas e nenhum comprometimento silencioso — de forma robusta entre níveis de modelo trabalhador, sem que daí decorra qualquer vantagem de acurácia de detecção.
3. **Uma metodologia de avaliação bidimensional e um arcabouço reproduzível.** O trabalho entrega uma matriz de avaliação adversarial — corpus de 185 *playbooks* LOLBin, injeções em quatro modos e três níveis, gatilho de saúde de telemetria, extração priorizada de EID1 e pontuação determinística por janela com intervalos de confiança por *bootstrap* — que pontua os agentes não apenas pelo acerto da classificação, mas também pela capacidade de detectar que foram atacados. Essa dupla dimensão de avaliação é, ela própria, uma contribuição metodológica transferível para a pesquisa em agentes de segurança.

8.4 Trabalhos Futuros

A evidência reunida e as limitações reconhecidas apontam cinco direções de continuidade:

1. **Saturar a telemetria discriminativa.** Reconfigurar a coleta de Sysmon e complementá-la com fontes adicionais (ETW, EDR de *endpoint*, telemetria de PowerShell e de rede) para elevar o teto estrutural de revocação imposto pela captura esparsa de EID1, isolando assim o efeito da governança da escassez de sinal. Só com a telemetria saturada será possível afirmar com segurança se a igualdade de F2 entre C1-full e C2-naive persiste.
2. **Estressar adversarialmente a detecção, e não apenas a observabilidade.** Conceber *payloads* de injeção que efetivamente alterem a evidência consumida pelo classificador (e não apenas o instruem a ignorá-la), de modo a testar se a resistência da governança se traduz em *melhor acurácia sob ataque forte* — regime em que o agente de passagem única tenderia a falhar silenciosamente.
3. **Reduzir a taxa de falsos positivos.** Incorporar calibração, agrupamento de alertas, validação determinística adicional e aprendizado com *feedback* do analista, transpondo o ponto de operação otimizado para F2 a um ponto operacionalmente viável em SOC.
4. **Ampliar o escopo de avaliação.** Estender o corpus a técnicas pós-exploração além de LOLBins e a ambientes Linux, macOS e nuvem pública, com validação longitudinal em redes de maior escala e diversidade de *background*.
5. **Separar o efeito do modelo do efeito da governança.** Reavaliar a arquitetura com diferentes modelos de linguagem subjacentes e medir custos operacionais (latência de triagem, consumo de *tokens*, custo financeiro e tempo até a validação de evidência), de modo a caracterizar o *trade-off* entre robustez de governança e custo computacional.

8.5 Considerações Finais

O MAS-Hunt não elimina a necessidade de SIEM, de telemetria confiável ou de validação humana; ele os pressupõe e sobre eles opera. A principal lição deste trabalho não é a que se esperava no início: a governança multiagente não tornou o sistema um detector mais acurado. O que ela tornou o sistema foi um detector mais *honesto* sob ataque — capaz de declarar, sob manipulação adversarial, que seu próprio raciocínio foi alvo de contaminação, em vez de produzir uma classificação aparentemente correta sem qualquer rastro do ataque que sofreu. Em segurança cibernética, na qual o adversário explora justamente a confiança cega no processo de decisão, essa diferença — entre errar (ou acertar) em silêncio e deixar evidência auditável da manipulação — é precisamente o que separa um agente conveniente de um agente defensável. É essa propriedade, e não um número de acurácia superior, que a evidência experimental desta dissertação sustenta como a contribuição de segurança do MAS-Hunt.

Referências

- A et al. 2024 A, P. J. et al. Threat hunting in system using log analysis. In: *2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS)*. [S.l.]: IEEE, 2024. p. 1–6.
- Agashe et al. 2025 AGASHE, S. et al. LLM-Coordination: Evaluating and analyzing multi-agent coordination abilities in large language models. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. [S.l.]: Association for Computational Linguistics, 2025. p. 8038–8057.
- Aldauji, Batarfi e Bayousef 2022 ALDAUIJI, F.; BATARFI, O.; BAYOUSEF, M. Utilizing cyber threat hunting techniques to find ransomware attacks: A survey of the state of the art. *IEEE Access*, v. 10, p. 61695–61706, 2022.
- Ali e Kostakos 2023 ALI, T.; KOSTAKOS, P. *HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs)*. 2023.
- Bai e Sugawara 2024 BAI, Y.; SUGAWARA, T. Reducing redundant computation in multi-agent coordination through locally centralized execution. In: *2024 16th IIAI International Congress on Advanced Applied Informatics (IIAI-AAI)*. [S.l.]: IEEE, 2024. p. 363–368.
- Brasil 2012 Brasil. *Lei n. 12.737, de 30 de novembro de 2012 (Lei Carolina Dieckmann)*. 2012. [Acesso em 12 fev. 2025]. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2011-2014/2012/lei/l12737.htm.
- Brasil 2014 Brasil. *Lei n. 12.965, de 23 de abril de 2014 (Marco Civil da Internet)*. 2014. [Acesso em 12 fev. 2025]. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2011-2014/2014/lei/l12965.htm.
- Brasil 2018 Brasil. *Lei n. 13.709, de 14 de agosto de 2018 (Lei Geral de Proteção de Dados Pessoais)*. 2018. [Acesso em 12 fev. 2025]. Disponível em: http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.
- Chen et al. 2024 CHEN, B. et al. BlockAgents: Towards byzantine-robust LLM-based multi-agent coordination via blockchain. In: *Proceedings of the ACM Turing Award Celebration Conference - China 2024 (ACM-TURC '24)*. Changsha, China: Association for Computing Machinery, 2024. p. 187–192.
- Chen et al. 2024 CHEN, Z. et al. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In: *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2024. v. 37, p. 130185–130213.
- Dolan-Gavitt 2025 DOLAN-GAVITT, B. *AI Agents for Offsec with Zero False Positives*. 2025. Black Hat USA 2025 Briefings, Las Vegas, NV. Disponível em: <https://i.blackhat.com/BH-USA-25/Presentations/US-25-Dolan-Gavitt-AI-Agents-for-Offsec-with-Zero-False-Positives-Thursday.pdf>.

- Durfee 1993 DURFEE, E. H. Organizations, plans and schedules: An interdisciplinary perspective on coordinating ai agents. *Journal of Intelligent Systems*, Walter de Gruyter, v. 3, n. 2-4, p. 157–188, 1993.
- Geiping et al. 2025 GEIPING, J. et al. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In: *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*. [s.n.], 2025. Spotlight. Disponível em: <https://openreview.net/forum?id=S3GhJooWIC>.
- Goldoni, Rodrigues e Medeiros 2024 GOLDONI, L. R. F.; RODRIGUES, K. F.; MEDEIROS, B. P. Qual é o futuro da governança de cibersegurança no brasil? *Cadernos Gestão Pública e Cidadania*, Fundação Getúlio Vargas, v. 29, p. e90972, 2024. Disponível em: <https://periodicos.fgv.br/cgpc/article/view/90972>.
- Gu et al. 2024 GU, X. et al. Agent smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. PMLR, 2024. (Proceedings of Machine Learning Research, v. 235), p. 16647–16672. Disponível em: <https://proceedings.mlr.press/v235/gu24e.html>.
- Guo et al. 2024 GUO, T. et al. Large language model based multi-agents: A survey of progress and challenges. In: *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. [S.l.]: International Joint Conferences on Artificial Intelligence Organization, 2024. p. 8048–8057.
- HaddadPajouh et al. 2018 HADDADPAJOUH, H. et al. A deep recurrent neural network based approach for internet of things malware threat hunting. *Future Generation Computer Systems*, Elsevier, v. 85, p. 88–96, 2018. ISSN 0167-739X.
- Hillier e Karroubi 2022 HILLIER, C.; KARROUBI, T. *Turning the Hunted into the Hunter via Threat Hunting: Life Cycle, Ecosystem, Challenges and the Great Promise of AI*. 2022.
- Huang et al. 2025 HUANG, J.-T. et al. On the resilience of LLM-based multi-agent collaboration with faulty agents. In: *Proceedings of the 42nd International Conference on Machine Learning (ICML)*. PMLR, 2025. (Proceedings of Machine Learning Research, v. 267), p. 26202–26226. Disponível em: <https://proceedings.mlr.press/v267/huang25ay.html>.
- ISACA 2025 ISACA. *Leveraging COBIT for Effective AI System Governance*. 2025. White Paper, ISACA. Disponível em: <https://www.isaca.org/resources/white-papers/2025/leveraging-cobit-for-effective-ai-system-governance>.
- Li et al. 2023 LI, Y. et al. Cooperative open-ended learning framework for zero-shot coordination. In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023. (Proceedings of Machine Learning Research, v. 202), p. 20470–20484. Disponível em: <https://proceedings.mlr.press/v202/li23au.html>.
- Liu et al. 2024 LIU, J. et al. LLM-Powered hierarchical language agent for real-time human-AI coordination. In: *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. [S.l.]: International Foundation for Autonomous Agents and Multiagent Systems, 2024. p. 1219–1228.

- Liu et al. 2024 LIU, Z. et al. CRDA: Content risk drift assessment of large language models through adversarial multi-agent interaction. In: *2024 International Joint Conference on Neural Networks (IJCNN)*. [S.l.]: IEEE, 2024. p. 1–8.
- Mahmud, Goldfajn e Zilberstein 2025 MAHMUD, S.; GOLDFAJN, D. B.; ZILBERSTEIN, S. *Distributed Multi-Agent Coordination Using Multi-Modal Foundation Models*. 2025.
- Mandiant 2025 Mandiant. *M-Trends 2025: Data, Insights, and Recommendations From the Frontlines*. [S.l.], 2025. 16th edition. Disponível em: <<https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2025>>.
- McIntosh et al. 2024 MCINTOSH, T. R. et al. From COBIT to ISO 42001: Evaluating cybersecurity frameworks for opportunities, risks, and regulatory compliance in commercializing large language models. *Computers & Security*, Elsevier, v. 144, p. 103964, 2024. ISSN 0167-4048.
- McIntosh et al. 2024 MCINTOSH, T. R. et al. From COBIT to ISO 42001: Evaluating cybersecurity frameworks for opportunities, risks, and regulatory compliance in commercializing large language models. *Computers & Security*, Elsevier, v. 144, p. 103964, 2024. ISSN 0167-4048.
- Moreno et al. 2025 MORENO, A. C. et al. Analysis of autonomous penetration testing through reinforcement learning and recommender systems. *Sensors*, MDPI, v. 25, n. 1, p. 211, 2025. ISSN 1424-8220.
- Oesch et al. 2025 OESCH, S. et al. The path to autonomous cyberdefense. *IEEE Security & Privacy*, v. 23, n. 1, p. 38–46, 2025. ISSN 1540-7993.
- Qiu et al. 2024 QIU, X. et al. *Towards Collaborative Intelligence: Propagating Intentions and Reasoning for Multi-Agent Coordination with Large Language Models*. 2024.
- Sindiramutty 2024 SINDIRAMUTTY, S. R. *Autonomous Threat Hunting: A Future Paradigm for AI-Driven Threat Intelligence*. 2024.
- Thi et al. 2022 THI, H. T. et al. Federated learning-based cyber threat hunting for apt attack detection in sdn-enabled networks. In: *2022 21st International Symposium on Communications and Information Technologies (ISCIT)*. [S.l.]: IEEE, 2022. p. 1–6.
- Ucci, Aniello e Baldoni 2019 UCCI, D.; ANIELLO, L.; BALDONI, R. Survey of machine learning techniques for malware analysis. *Computers & Security*, Elsevier, v. 81, p. 123–147, 2019. ISSN 0167-4048.
- Valencia 2024 VALENCIA, L. J. *Artificial Intelligence as the New Hacker: Developing Agents for Offensive Security*. 2024.
- Vaswani et al. 2017 VASWANI, A. et al. Attention is all you need. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*. [S.l.]: Curran Associates, Inc., 2017. p. 5998–6008.
- Verizon Business 2025 Verizon Business. *2025 Data Breach Investigations Report*. [S.l.], 2025. Disponível em: <<https://www.verizon.com/business/resources/reports/dbir/>>.

- Yi e Kim 2024 YI, C.-G.; KIM, Y.-G. Hypothesis generation model for cyber threat hunting. *IEEE Communications Magazine*, v. 62, n. 10, p. 110–116, 2024.
- Yuan et al. 2023 YUAN, L. et al. Robust multi-agent coordination via evolutionary generation of auxiliary adversarial attackers. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.]: AAAI Press, 2023. v. 37, n. 10, p. 11753–11762.
- Yulianto e Ngo 2024 YULIANTO, S.; NGO, G. N. C. Automated threat hunting, detection, and threat actor profiling using tira. In: *2024 International Conference on ICT for Smart Society (ICISS)*. [S.l.]: IEEE, 2024. p. 1–7.
- Zhan et al. 2024 ZHAN, Q. et al. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024. p. 10471–10506.
- Zhang et al. 2025 ZHANG, B. et al. Breaking agents: Compromising autonomous LLM agents through malfunction amplification. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Suzhou, China: Association for Computational Linguistics, 2025. p. 34964–34976.
- Zhang et al. 2024 ZHANG, C. et al. ProAgent: Building proactive cooperative agents with large language models. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.]: AAAI Press, 2024. v. 38, n. 16, p. 17591–17599.